



Motor de Jogo 2D Modular em C++23 com OpenGL e SDL3

Projeto Final de Licenciatura

Elaborado por:

Guilherme Matos- nº de aluno 202327111

Orientador:

Rui Miguel Pascoal

Barcarena

setembro, 2026

Gestão de Sistemas e Computação

Motor de Jogo 2D Modular em C++23 com OpenGL e SDL3

Projeto Final de Licenciatura

Elaborado por:

Guilherme Matos- nº de aluno 202327111

Orientador:

Rui Miguel Pascoal

Barcarena

setembro, 2026

O autor é o único responsável pelas ideias expressas neste relatório

Resumo

O presente relatório documenta o desenvolvimento de um motor de jogo 2D modular, implementado no standard C++23, com recurso às bibliotecas SDL3 (para gestão de janela, input, áudio e eventos) e OpenGL Core Profile 4.6 (para renderização acelerada por hardware), complementadas pelo carregador GLAD, pela biblioteca de ECS EnTT e pela biblioteca stb_image para carregamento de texturas. O projeto foi planeado durante 2-3 meses e desenvolvido durante 2 meses, realizado no âmbito da unidade curricular de Projeto de Software.

Durante os três meses de planeamento, originalmente o projeto passou por diferentes ideias, o primeiro objetivo inicial foi um modelo LLM desenvolvido e treinado de raiz. No entanto, o custo de oportunidade revelou-se consideravelmente elevado em custos energéticos e em hardware devido ao seu aumento do custo elevado (memórias RAM).

A segunda ideia foi o desenvolvimento de uma pequena rede montada com servidores com várias funções (Active Directory Domain Controller, DNS, DHCP, Controlos de Acesso, etc.).

A motivação central para a ideia final, do Game Engine prende-se com a necessidade de compreender, na prática, os princípios arquiteturais que sustentam motores comerciais como Unity, Unreal Engine ou Godot. Enquanto a maioria dos recursos disponíveis foca-se na utilização de motores existentes, este projeto propõe-se a construir um motor de raiz, confrontando o autor com decisões arquiteturais fundamentais: como estruturar o loop principal, como organizar as entidades de jogo, como desenhar o pipeline de renderização e como desacoplar os subsistemas.

O objetivo geral foi o de desenvolver uma base de código reutilizável, extensível e bem documentada, que abstraia as camadas de baixo nível e permita a criação rápida de jogos 2D com uma arquitetura baseada em Entidades-Componentes-Sistemas (ECS). Para o efeito, foram definidos objetivos específicos que incluíram a implementação de módulos independentes para áudio, gráficos, eventos, ECS, input e recursos; a adoção do padrão ECS com a biblioteca EnTT; a implementação de um sistema de renderização 2D com suporte a

texturas, tiling e transparência; e o desenvolvimento de uma cena de demonstração que validasse a integração de todos os subsistemas.

Os resultados obtidos demonstram a viabilidade da abordagem, testados com um desempenho estável a 60 FPS e 144FPS, dependente da taxa de atualização do monitor utilizado e uma cena funcional que integra todos os subsistemas. A arquitetura final é composta por módulos independentes, demonstrando os princípios de separação de responsabilidades e código desacoplado. As principais dificuldades encontradas - controlos de câmara e renderização dos cenários de jogo - estão documentadas e as suas soluções descritas em detalhe. Como trabalho futuro, propõe-se a implementação de um sistema de animação de sprites, integração com um motor de física mais robusto, suporte a tilemaps e otimização do RenderSystem com batching.

ECS; OpenGL; SDL3; Multi-plataforma; Arquitetura Modular

Abstract

This report documents the development of a modular 2D game engine, implemented in C++23 standard, using the SDL3 library (for window management, input, audio and events) and OpenGL Core Profile 4.6 (for hardware-accelerated rendering), complemented by the GLAD loader, the EnTT ECS library and the stb_image library for texture loading. The project was planned over 2–3 months and developed over 2 months, carried out within the scope of the Software Project course.

During the three months of planning, the project originally went through different ideas. The first initial goal was to be an LLM model developed and trained from scratch. However, the opportunity cost proved to be somewhat high in terms of energy and hardware costs due to the excessive increase in cost (RAM memory). The second idea was the development of a small network assembled with servers with various functions (Active Directory Domain Controller, DNS, DHCP, Access Controls, etc.).

The central motivation for the final idea, the Game Engine, stems from the need to understand, in practice, the architectural principles that support commercial engines such as Unity, Unreal Engine or Godot. While most available resources focus on using existing engines, this project proposes to build an engine from scratch, confronting the author with fundamental architectural decisions: how to structure the main loop, how to organize game entities, how to design the rendering pipeline, and how to decouple subsystems.

The general objective was to develop a reusable, extensible and well-documented codebase that abstracts the low-level layers and enables the rapid creation of 2D games with an architecture based on Entity-Component-System (ECS). To this end, specific objectives were defined, which included the implementation of independent modules for audio, graphics, events, ECS, input and resources; the adoption of the ECS pattern with the EnTT library; the implementation of a 2D rendering system with support for textures, tiling and transparency; and the development of a demonstration scene that would validate the integration of all subsystems.

The results obtained demonstrate the viability of the approach, tested with stable performance at 60FPS and 144FPS, based on the used monitor refresh rate and a functional scene that

integrates all subsystems. The final architecture is composed of independent modules, demonstrating the principles of role separation and code decoupling. The main difficulties encountered – camera controls and rendering of game scenes – are documented and their solutions described in detail. As future work, the implementation of a Sprite animation system, integration with a more robust physics engine, tilemap support and optimization of the RenderSystem with batching are proposed.

ECS ; OpenGL; SDL3; Cross-platform; Modular Architecture

Índice

1. Introdução.....	1
1.1 Contexto e Enquadramento.....	1
1.2 Motivação.....	2
1.3 Problemas.....	3
1.4 Objetivos.....	3
1.4.1 Objetivo Geral.....	3
1.4.2 Objetivos Específicos.....	4
1.5 Metodologia Adotada (Breve Descrição).....	5
1.6 Estrutura do Documento.....	5
2. Estado da Arte / Enquadramento.....	7
2.1 Introdução ao Capítulo.....	7
2.2 Motores de Jogo 2D.....	7
2.2.1 Definição e Componentes.....	7
2.2.2 Evolução Histórica.....	9
2.2.3 Motores 2D Relevantes.....	9
2.2.4 Síntese e Implicações.....	10
2.3 A Biblioteca SDL.....	10
2.3.1 Descrição Geral.....	10

2.3.2 Evolução: SDL2 vs SDL3.....	10
2.3.3 Papel no Presente Projeto.....	11
2.3.4 Síntese e Implicações.....	12
2.4 A API OpenGL.....	12
2.4.1 Descrição Geral.....	12
2.4.2 OpenGL Moderno vs Legacy.....	12
2.4.3 Pipeline de Renderização.....	13
2.4.4 GLAD – OpenGL Loader.....	14
2.4.5 Síntese e Implicações.....	14
2.5 O Padrão Entidades-Componentes-Sistemas (ECS).....	15
2.5.1 Definição.....	15
2.5.2 Vantagens do ECS.....	16
2.5.3 EnTT — Detalhes.....	16
2.5.4 Síntese e Implicações.....	17
2.6 Padrões de Design Aplicáveis.....	17
2.6.1 Game Loop.....	17
2.6.2 Singleton.....	17
2.6.3 Observer / Event Bus.....	18
2.6.4 State / Scene Stack.....	18
2.6.5 Component / ECS.....	18

2.6.6 Síntese e Implicações.....	18
2.7 Sistemas de Áudio em Motores de Jogo.....	18
2.7.1 Desafios do Áudio.....	18
2.7.2 Abordagem do Presente Projeto.....	19
2.8 Sistemas de Eventos em Motores de Jogo.....	19
2.8.1 Necessidade de Desacoplamento.....	19
2.8.2 Abordagem do Presente Projeto.....	19
2.9 Comparação de Abordagens e Síntese Final.....	20
3. Metodologia.....	21
3.1 Abordagem Metodológica.....	21
3.1.1 Contexto e desafios.....	21
3.1.2 Metodologia adotada.....	21
3.1.3 Pilares da Abordagem Adotada.....	22
3.1.3.1 Desenvolvimento Iterativo e Incremental.....	22
3.1.3.2 Desenvolvimento Exploratório com Git.....	23
3.1.3.3 Adaptação contínua.....	24
3.2 Plano de Avaliação.....	25
3.2.1 Testes de Integração Manuais.....	25
3.2.2 Análise de Logs.....	25
3.2.3 Métricas de Desempenho.....	26

3.2.4 Inspeção Visual.....	26
3.2.5 Análise Estática.....	26
3.3 Ferramentas Utilizadas.....	26
3.4 Cronograma e Fases.....	28
3.5 Dificuldades Encontradas.....	30
3.5.1 Detalhe das Dificuldades Mais Relevantes.....	31
4. Especificação de Requisitos.....	32
4.1 Introdução.....	32
4.2 Requisitos Funcionais.....	32
4.2.1 Detalhe dos Requisitos Funcionais.....	33
4.3 Requisitos Não-Funcionais.....	35
4.4 Casos de Uso.....	35
4.4.1 Caso de Uso 1 — Iniciar o Motor.....	35
4.4.2 Caso de Uso 2 — Movimentar o Jogador.....	36
4.4.3 Caso de Uso 3 — Saltar em Cima de Inimigo.....	36
4.5 Restrições e Pressupostos.....	36
4.5.1 Restrições.....	36
4.5.1.1 Restrições Temporais.....	36
4.5.1.2 Restrições de Escopo.....	37
4.5.2 Pressupostos.....	38

4.5.2.1 Pressupostos de Ambiente de Execução.....	38
4.5.2.2 Pressupostos de Utilização.....	39
4.5.2.3 Pressupostos Técnicos.....	39
4.5.2.4 Pressupostos de Manutenção.....	40
5. Implementação.....	41
5.1 Introdução ao Capítulo.....	41
5.2 Arquitetura Global.....	43
5.2.1 Visão Geral.....	43
5.2.2 Princípios Arquiteturais.....	44
5.2.3 Fluxo de Execução.....	45
5.2.4 Dependências.....	46
5.3 Módulo core.....	46
5.3.1 Descrição.....	46
5.3.2 Application.....	47
5.3.3 Engine.....	49
5.3.4 Time.....	50
5.3.5 Input.....	50
5.3.6 GamepadManager.....	51
5.3.7 PlayerController.....	51
5.3.8 CameraController.....	52

5.3.9 FpsCounter.....	53
5.4 Módulo graphics.....	53
5.4.1 Descrição.....	53
5.4.2 GraphicsContext.....	54
5.4.3 Shader.....	54
5.4.4 Texture2D.....	55
5.4.5 Font.....	56
5.5 Módulo renderer.....	56
5.5.1 Descrição.....	56
5.5.2 Implementação.....	57
5.5.3 Decisões de Design.....	57
5.5.4 Fluxo de Renderização.....	58
5.6 Módulo scene.....	58
5.6.1 Descrição.....	58
5.6.2 Scene e SceneManager.....	59
5.6.3 Camera2D.....	60
5.6.4 GameWorld.....	61
5.6.5 ECSScene.....	61
5.7 Módulo systems.....	61
5.7.1 Descrição.....	61

5.7.2 RenderSystem.....	62
5.7.3 EnemySystem.....	63
5.8 Módulo components.....	64
5.8.1 Descrição.....	64
5.8.2 Implementação.....	64
5.9 Módulo audio.....	65
5.9.1 Descrição.....	65
5.9.2 SoundEffect.....	66
5.9.3 SoundManager.....	66
5.9.4 Dificuldade Encontrada.....	66
5.10 Módulo events.....	66
5.10.1 Descrição.....	66
5.10.2 EventBus.....	67
5.10.3 Fluxo de um Evento.....	68
5.11 Módulo resources.....	68
5.11.1 Descrição.....	68
5.11.2 ResourceManager.....	68
5.12 Módulo diagnostics.....	70
5.12.1 Descrição.....	70
5.12.2 Log.....	70

5.12.3 Assert.....	71
5.13 Módulo math.....	71
5.13.1 Descrição.....	71
5.13.2 Matrix4.....	72
5.14 Cena de Demonstração (Demo2DFixedScene).....	73
5.14.1 Descrição.....	73
5.14.2 Estrutura.....	73
5.14.3 SetupScene.....	74
5.14.4 Fluxo de Atualização.....	74
5.14.5 CheckEnemyCollisions.....	75
5.15 Menu Principal (MenuScene).....	76
5.15.1 Descrição.....	76
5.15.2 Criação de Texto.....	76
5.15.3 Clique do Rato.....	77
5.16 CMake e Build System.....	77
5.16.1 Estrutura.....	77
5.16.2 Configuração Inicial.....	78
5.16.3 Cópia de DLLs.....	79
5.16.4 Comandos de Build.....	79
5.17 Tratamentos de erro e Robustez.....	79

5.17.1 Estratégia Adotada.....	79
5.17.2 Tipos de erro.....	80
5.17.3 Logging como Ferramenta de Diagnóstico.....	80
6. Resultados e Discussão.....	82
6.1 Introdução.....	82
6.2 Resultados Obtidos.....	82
6.2.1 Estado Final do Motor.....	82
6.2.2 Desempenho.....	84
6.2.3 Cumprimento dos Requisitos Funcionais.....	84
6.2.4 Cumprimento dos Requisitos Não-Funcionais.....	85
6.3 Análise Crítica.....	85
6.3.1 Pontos Fortes.....	85
6.3.2 Pontos Fracos.....	86
6.3.3 Fatores de Sucesso.....	87
6.3.4 Bloqueios e Dificuldades.....	88
6.4 Comparação com o Estado da Arte.....	89
6.4.1 Comparação com Motores Comerciais.....	89
6.4.2 Comparação com Projetos Académicos Similares.....	90
6.5 Limitações.....	91
6.5.1 Limitações Técnicas.....	91

6.5.2 Limitações de Escopo.....	92
6.6 Ameaças à validade.....	94
6.6.1 Ameaças Internas.....	94
6.6.2 Ameaças Externas.....	94
6.7 Lições Aprendidas.....	95
6.7.1 Lições Técnicas.....	95
6.7.2 Lições Metodológicas.....	96
7. Conclusão e Trabalho Futuro.....	97
7.1 Síntese do Trabalho.....	97
7.2 Contribuições.....	98
7.2.1 Contribuições Técnicas.....	98
7.2.2 Contribuições Documentais.....	99
7.2.3 Contribuições Metodológicas.....	100
7.2.4 Contribuições para Trabalho Futuro.....	100
7.3 Limitações Reconhecidas.....	100
7.4 Trabalho Futuro.....	101
7.4.1 Curto Prazo (Alta Prioridade).....	101
7.4.2 Médio Prazo (Prioridade Média).....	102
7.4.3 Longo Prazo (Baixa Prioridade).....	102
7.5 Reflexão Final.....	103

Referências.....	104
Apêndices.....	105
Apêndice A — Excertos de Código Relevantes.....	105
A.1 — Application::Initialize.....	105
A.2 — RenderSystem::Render.....	110
A.3 — EnemySystem::Update.....	112
A.4 — PlayerController::Update.....	116
A.5 — Demo2DFixedScene::SetupScene.....	121
A.6 — MenuScene::Update.....	127
Apêndice B — Diagrama de Classes.....	130
Anexos.....	131
Anexo A — Documentação Doxygen Gerada.....	131
Anexo B — Instruções de Compilação.....	131
Windows.....	131
Linux.....	131
Anexo C — Referências Consultadas Online.....	131

Índice de figuras

Figura 1- Exemplo de combate com vista isométrica em Final Fantasy Tactics (Square, 2025)	2
Figura 2- Overview dos subsistemas utilizados em Game Engines. Gregory, J. (2018)	8
Figura 3- Criação de um Sprite	15
Figura 4- Componente Transform	16
Figura 5 - Estrutura do projeto	43
Figura 6 - Função Initialize()	48
Figura 7 - Função Run()	49
Figura 8 - Função Update() da classe PlayerController	52
Figura 9 - Compilação de Shaders	55
Figura 10 - Função DrawTexturedQuad()	57
Figura 11 - Classe Scene	59
Figura 12 - Função Update() da classe Camera2D	60
Figura 13 - Método Render() da classe RenderSystem	62
Figura 14 - Método Update() da classe EnemySystem	63
Figura 15 - Captura de eventos e seu handling	67
Figura 16 - Função SetupScene() para criação da cena	74
Figura 17 - Detecção de colisões com a função CheckEnemyCollisions()	75

Figura 18 - Renderização de texto.....	76
Figura 19 - Captura do clique do rato.....	77
Figura 20- Diagrama UML de classes.....	130

Índice de tabelas

Tabela 1 - Comparação entre OpenGL Legacy vs Core.....	13
Tabela 2 - Comparação SDL2 vs SDL3.....	20
Tabela 3 - Versionamento das ferramentas utilizadas.....	27
Tabela 4 - Dificuldades encontradas.....	30
Tabela 5 - Requisitos funcionais.....	32
Tabela 6 - Requisitos não funcionais.....	35
Tabela 7- Classificação de erros.....	80
Tabela 8-Estado final dos subsistemas do motor.....	83
Tabela 9- Comparação com motores comerciais.....	89

Lista de Abreviaturas e Siglas

Sigla	Significado
AABB	Axis-Aligned Bounding Box
API	Application Programming Interface
CMake	Cross-platform Make
CPU	Central Processing Unit
DLL	Dynamic-Link Library
DOD	Data-Oriented Design
DOI	Digital Object Identifier
ECS	Entity-Component-System
FPS	Frames Per Second
GLAD	OpenGL Loader Generator
GLSL	OpenGL Shading Language
GPU	Graphics Processing Unit

Sigla	Significado
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
PCM	Pulse-Code Modulation
RQ	Research Question
SDL	Simple DirectMedia Layer
TTF	TrueType Font
UML	Unified Modeling Language
WAV	Waveform Audio File Format
WIP	Work In Progress

1. Introdução

1.1 Contexto e Enquadramento

O desenvolvimento de videojogos é uma área exigente e interdisciplinar da engenharia de software.

Combina desafios de tempo real, gestão rigorosa de recursos limitados (memória, CPU, GPU), sincronização de eventos, e uma necessidade constante de iterar sobre ideias de jogo sem comprometer a estabilidade do sistema.

Neste contexto, os motores de jogo surgem como plataformas fundamentais, abstraindo as complexidades de baixo nível (renderização, áudio, input, gestão de recursos) e oferecendo aos programadores de jogo ferramentas de alto nível para expressar a lógica, a estética e o conteúdo do jogo.

A indústria de videojogos movimenta, globalmente, mais de 180 mil milhões de dólares anuais (Newzoo, 2024), sendo uma das maiores indústrias de entretenimento do mundo. Este peso económico traduz-se numa procura crescente de profissionais com competências sólidas em programação gráfica, arquitetura de software e otimização de desempenho. Neste panorama, a compreensão profunda de como um motor de jogo funciona internamente é uma vantagem competitiva significativa para qualquer engenheiro de software.

Historicamente, os primeiros motores de jogo surgiram nos anos 90, com soluções como o *Doom Engine* (id Software, 1993), o *Quake Engine* e o seu sucessor, *id Tech* (id Software, 1996). A evolução destes motores levou ao aparecimento de soluções comerciais como Unity (2005), Unreal Engine (Epic Games, 1998) – licenciamento em código fechado, originalmente utilizado para a série de jogos Unreal - e Godot (2014) – licenciamento Open-Source -, embora motores diferentes, partilham princípios arquiteturais comuns: modularidade, separação de responsabilidades, e um pipeline de renderização bem definido.

O presente projeto insere-se neste contexto, propondo-se a construir, de raiz, um motor de jogo 2D modular em C++23, utilizando SDL3 para a camada de plataforma e OpenGL 4.6 Core Profile para a renderização. O foco não é competir com motores comerciais, mas sim compreender e aplicar, num contexto académico, os princípios arquiteturais que os sustentam.

No caso de continuidade de desenvolvimento do projeto após conclusão da unidade curricular, o objetivo posterior será manter e expandir o motor, e a distribuição será feita com licenciamento de código aberto – ainda não decidido entre licença GNU-GPLv3 ou zlib.

1.2 Motivação

A primeira motivação que justifica o desenvolvimento deste projeto é uma motivação pedagógica: os motores comerciais, embora poderosos, escondem os seus detalhes internos atrás de abstrações de alto nível. Trabalhar com o Unity ou o Godot ensina a usar um motor, mas não ensina como um motor funciona. Ao construir um motor de raiz, o programador é forçado a confrontar-se com decisões arquiteturais fundamentais: como gerir o loop principal? Como estruturar as entidades de jogo? Como organizar o pipeline de renderização? Como desacoplar os subsistemas?

A segunda motivação, após finalização da unidade curricular, será dar continuidade ao desenvolvimento deste projeto e eventualmente desenvolver um jogo (possivelmente do género *RPG* ou *Roguelike* com vista isométrica, ex: *Final Fantasy Tactics* (Figura 1)) utilizando as funcionalidades desenvolvidas ao longo deste projeto.



Figura 1- Exemplo de combate com vista isométrica em Final Fantasy Tactics (Square, 2025)

1.3 Problemas

Apesar da abundância de recursos online sobre programação de jogos, existe alguma falta na documentação transparente do processo de desenvolvimento de um motor de jogo. A maioria dos recursos disponíveis foca-se em:

Tutoriais de utilização de motores existentes (Unity, Godot, Unreal);

Tutoriais pontuais sobre uma técnica específica (ex: como desenhar um triângulo em OpenGL);

Documentação de API de bibliotecas (SDL, OpenGL, etc.);

O que falta é uma narrativa completa que documente as decisões arquiteturais, as dificuldades encontradas, as soluções adotadas e as limitações reconhecidas ao longo do desenvolvimento de um motor de jogo. Para o desenvolvimento de um projeto semelhante, a informação necessária relevante é maioritariamente encontrada muito espalhada e incompleta em fóruns e canais de comunicação de perguntas e respostas como o Reddit, Stack Overflow, tornando difícil encontrar alguma referência completa que demonstra o caso completo de início ao fim.

Adicionalmente, muitos projetos académicos apresentam apenas o resultado final, omitindo o percurso - incluindo as tentativas falhadas e as correções. Esta omissão empobrece a aprendizagem e não reflete a realidade da engenharia de software, que é inerentemente iterativa e repleta de trade-offs.

1.4 Objetivos

1.4.1 Objetivo Geral

Desenvolver um motor de jogo 2D modular em C++23, com recurso a SDL3 e OpenGL 4.6, que sirva de base reutilizável para a criação de jogos 2D e que demonstre a aplicação de princípios arquiteturais sólidos.

1.4.2 Objetivos Específicos

O desenvolvimento deste motor requer algumas especificidades durante o seu percurso sendo eles:

- Implementação de uma arquitetura modular com separação clara de responsabilidades, organizada em módulos independentes (áudio, gráficos, eventos, ECS, input, recursos, etc.);
- Adoção do padrão Entidades-Componentes-Sistemas (ECS) para a gestão de objetos de jogo - neste caso foi utilizado a biblioteca EnTT;
- Implementação do sistema de renderização 2D com suporte a texturas, tiling, transparência e renderização de texto com a biblioteca SDL_ttf;
- Implementação de sistema de áudio com suporte a efeitos sonoros WAV e conversão automática de formatos;
- Implementação de um sistema de eventos desacoplado, baseado num EventBus centralizado;
- Implementar sistema de gestão de cenas com transições controladas;
- Desenvolvimento de cena para demonstração que valide a integração de todos os subsistemas, incluindo um pequeno plataforma com patrulha de inimigos, colisões AABB e mecânica de stomping;
- Documentação de todo o código fonte – foi utilizado o Doxygen para produzir documentação HTML navegável;
- Documentar o processo de desenvolvimento neste relatório, seguindo as normas APA 7;
- Estabelecer uma base sólida para trabalho futuro, identificando limitações e propondo melhorias concretas.

1.5 Metodologia Adotada (Breve Descrição)

Foi seguida uma abordagem mais pragmática e adaptativa, focada em desenvolvimento exploratório, utilizando o Git, baseado em três fatores muito importantes:

- **Desenvolvimento Iterativo e Incremental** - organização do trabalho em incrementos semanais, cada um com um objetivo concreto e validável através de uma demo funcional na cena de demonstração;
- **Desenvolvimento Exploratório com Git** - implementação direta no repositório principal no GitHub, com o Git a servir como ferramenta de suporte, segurança e rastreabilidade, e a facilidade de desenvolvimento em diferentes computadores. Cada funcionalidade era iterativamente refinada até funcionar, e após sucesso, foi feito o commit. O histórico de commits documenta o progresso;
- **Adaptação Contínua** - repriorização semanal com base nas dificuldades encontradas, no conhecimento adquirido.

1.6 Estrutura do Documento

Este relatório está organizado em sete capítulos, cada um respondendo a uma pergunta específica:

Capítulo 1 - Introdução: contextualiza o problema, apresenta a motivação e define os objetivos, e finaliza com uma descrição breve para cada capítulo.

Capítulo 2 - Estado da Arte / Enquadramento: revê a literatura relevante sobre motores de jogo, ECS, SDL, OpenGL e padrões de design, terminando com uma síntese das implicações para o projeto.

Capítulo 3 - Metodologia: descreve a abordagem metodológica adotada, as ferramentas utilizadas, o plano de avaliação e as dificuldades encontradas.

Capítulo 4 - Especificação de Requisitos: enumera os requisitos funcionais e não-funcionais do motor.

Capítulo 5 - Implementação: detalha a arquitetura global, o sistema ECS, o pipeline de renderização, o sistema de áudio, o sistema de eventos e a cena de demonstração.

Capítulo 6 - Resultados e Discussão: apresenta os resultados obtidos, analisa criticamente as limitações e discute ameaças à validade.

Capítulo 7 - Conclusão e Trabalho Futuro: sintetiza as contribuições, reconhece limitações e propõe desenvolvimentos futuros.

Complementam o documento as secções de Referências, Apêndices e Anexos.

2. Estado da Arte / Enquadramento

2.1 Introdução ao Capítulo

Este capítulo revê a literatura relevante para o projeto, organizada em cinco grandes temas: **motores de jogo 2D, a biblioteca SDL, a API OpenGL, o padrão ECS, e padrões de design aplicáveis**. O objetivo não é fazer uma listagem exaustiva de todos os trabalhos existentes, mas sim **identificar as abordagens relevantes, compará-las criticamente e extrair implicações concretas** para as decisões técnicas do projeto.

A revisão segue, portanto, uma estrutura temática, não cronológica. Cada secção termina com uma síntese e com as implicações para o projeto.

2.2 Motores de Jogo 2D

2.2.1 Definição e Componentes

Um motor de jogo é um *framework* de software que abstrai as complexidades de baixo nível do desenvolvimento de jogos, oferecendo um conjunto de subsistemas reutilizáveis: renderização, animações, física, áudio, input, gestão de recursos, inteligência artificial, *networking*, e gestão de cenas.

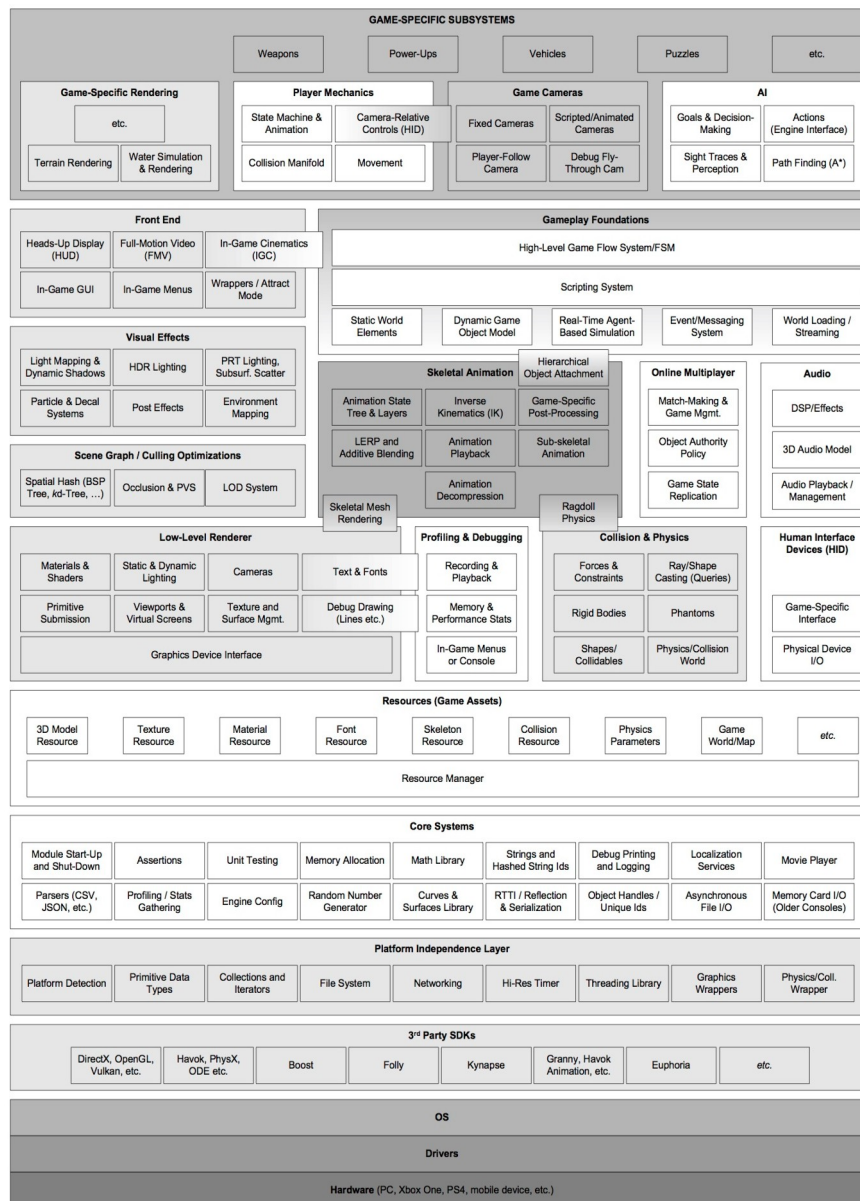


Figura 2- Overview dos subsistemas utilizados em Game Engines. Gregory, J. (2018)

Esta organização em camadas é um princípio arquitetural recorrente nos motores modernos, promovendo separação de responsabilidades, testabilidade e substituíbilidade.

2.2.2 Evolução Histórica

A evolução dos motores de jogo 2D acompanha a evolução do hardware e das linguagens de programação.

Anos 80 – Motores monolíticos: cada jogo era um sistema monolítico, com código altamente acoplado ao hardware específico. Não existia o conceito de “motor” reutilizável, o código era escrito de raiz para cada jogo.

Anos 90 – Primeiros motores reutilizáveis: surgem os primeiros motores verdadeiramente reutilizáveis, como o *Build Engine* (1995) e o *Doom Engine* (1993). Estes motores introduziram conceitos como BSP (*Binary Space Partitioning*) e renderização por *software*.

Anos 2000 – Consolidação e comercialização: o aparecimento de Unity (2005) e a evolução do Unreal Engine (1998) comercializam os motores de jogo, tornando-os acessíveis a estúdios independentes. Surge a distinção entre motores *AAA* (Unreal, CryEngine) e motores indie (GameMaker, Construct).

Anos 2010 – Open-source e comunidade: a abertura do código fonte do Godot (2014) e a evolução do LibGDX (2010) e do Phaser (2013) democratizam ainda mais o acesso a motores de qualidade profissional.

Anos 2020 – Especialização e ECS: a adoção generalizada do padrão ECS (EnTT, Flecs, Bevy) e a evolução do C++ moderno permitem motores mais otimizadores e modulares.

2.2.3 Motores 2D Relevantes

Godot Engine: motor de código open-source com suporte a 2D e 3D, escrito em C++ e com suporte C# oficial para programação de lógica de jogo. A versão 4.x introduziu um sistema ECS, com excelente integração de editores visuais. A sua arquitetura é uma referência importante para o presente projeto.

Unity 2D: solução comercial com vasta documentação e comunidade. A arquitetura baseada em *GameObjects* e *Components* é conceptualmente semelhante ao ECS, embora com diferenças significativas de desempenho.

Bevy Engine: motor emergente escrito em Rust, com uma arquitetura customizada puramente ECS. A sua abordagem *data-oriented* é uma referência teórica importante, embora a linguagem seja diferente.

2.2.4 Síntese e Implicações

A literatura sugere que um motor de jogo 2D moderno deve:

- Adotar uma arquitetura em camadas, com separação clara de responsabilidades;
- Utilizar um padrão de composição - ECS - em vez de herança profunda;
- Oferecer uma API de alto nível que abstraia as complexidades de baixo nível;
- Ser extensível e reutilizável, sem sacrificar o desempenho.

O presente projeto adota estas premissas, optando por uma arquitetura modular com ECS.

2.3 A Biblioteca SDL

2.3.1 Descrição Geral

A **Simple DirectMedia Layer** (SDL) é uma biblioteca *cross-platform* de baixo nível que fornece acesso a áudio, teclado, rato, joystick e hardware gráfico (através de OpenGL/Direct3D/Vulkan/Metal). É escrita em C, com *bindings* para várias linguagens, e é amplamente utilizada na indústria de jogos e em aplicações multimédia.

2.3.2 Evolução: SDL2 vs SDL3

A versão 2 (SDL2), lançada em 2013, tornou-se o padrão de facto para desenvolvimento de jogos *cross-platform* em C/C++. A versão 3 (SDL3), lançada em 2025, introduz várias melhorias significativas:

- **API de áudio reformulada:** introdução de `SDL_AudioStream`, que simplifica a conversão de formatos e a gestão de *buffers*;
- **Novo sistema de eventos:** maior consistência e melhor desempenho;

- **Suporte a *gamepads*:** API unificada para controladores;
- **Integração com Vulkan/Direct3D 12:** suporte de primeira classe a APIs gráficas modernas;
- **Melhor suporte a *high-DPI*:** gestão automática de escalas;
- **Suporte para o protocolo gráfico Wayland:** protocolo com objetivo de substituir o protocolo X11, recentemente tornou-se o mais utilizado pela maior parte dos *Desktop Environments* mais utilizados em várias distribuições Linux - GNOME, KDE Plasma – como também *Window Managers* como o Hyprland.

O presente projeto utiliza SDL3 pelas suas melhorias na API de áudio e no suporte a *gamepads* e principalmente o suporte ao protocolo *Wayland*, devido ao projeto ser desenvolvido principalmente num computador com Linux.

2.3.3 Papel no Presente Projeto

No presente projeto, SDL3 é responsável por:

- **Criação e gestão da janela** (SDLWindow);
- **Criação do contexto OpenGL** (SDL_GL_CreateContext);
- **Carregamento de funções OpenGL** via GLAD (gladLoadGLLoader);
- **Gestão de eventos** (teclado, rato, gamepad, janela);
- **Reprodução de áudio** (SoundEffect);
- **Carregamento de fontes TTF** (SDL_ttf).

Esta separação é crucial: SDL3 trata da plataforma, OpenGL trata da renderização, e o motor constrói a lógica de jogo sobre estas fundações.

2.3.4 Síntese e Implicações

SDL3 é uma escolha sólida para a camada de plataforma, oferecendo:

- **Portabilidade** (Windows, Linux, macOS, Android, iOS);
- **Maturidade** e documentação abundante;
- **API de áudio moderna**, adequada para efeitos sonoros e música;
- **Comunidade ativa** e evolução constante.

2.4 A API OpenGL

2.4.1 Descrição Geral

O **OpenGL** (Open Graphics Library) é uma API de renderização 2D e 3D, *cross-platform*, mantida pelo Khronos Group. Foi originalmente lançada em 1992 pela Silicon Graphics e tornou-se o padrão de facto para renderização acelerada por hardware em aplicações científicas, CAD e jogos.

2.4.2 OpenGL Moderno vs Legacy

O OpenGL passou por uma transformação significativa a partir da versão 3.2, com a introdução do conceito de **Core Profile**. Esta mudança eliminou as funcionalidades obsoletas (como o *fixed-function pipeline*) e introduziu um pipeline totalmente programável baseado em shaders.

Tabela 1 - Comparação entre OpenGL Legacy vs Core

Característica	Legacy (≤ 2.1)	Core (≥ 3.2)
Pipeline	Fixo (glBegin/glEnd)	Programável (shaders)
Vertex data	Imediato	Buffers (VBO/VAO)
Shaders	Não	Obrigatórios
Performance	Baixa	Alta
Portabilidade	Ampla	Moderna

O presente projeto utiliza **OpenGL 4.6 Core Profile**, a versão mais recente e madura lançada pela Khronos Group em 2016, redirecionando o foco para o desenvolvimento da API Vulkan.

2.4.3 Pipeline de Renderização

O pipeline OpenGL moderno pode ser resumido nas seguintes etapas:

- **Vertex Specification:** dados de vértices são enviados para a GPU via VBO;
- **Vertex Shader:** cada vértice é transformado por matrizes de modelo, vista e projeção;
- **Primitive Assembly:** vértices são agrupados em primitivas (triângulos, linhas);

- **Rasterization:** primitivas são convertidas em fragmentos;
- **Fragment Shader:** cada fragmento é colorido (ex: amostragem de textura);
- **Per-Sample Operations:** blending, depth test, stencil test;
- **Framebuffer:** resultado final é escrito no *framebuffer*.

O motor implementa este pipeline através das classes Shader, VertexBuffer, VertexArray e IndexBuffer.

2.4.4 GLAD – OpenGL Loader

Como o OpenGL é uma API cujas funções são carregadas dinamicamente em tempo de execução (via `wglGetProcAddress` no Windows, `glXGetProcAddress` no Linux), é necessário um **loader** que resolva os endereços das funções. O **GLAD** (OpenGL Loader Generator) é uma ferramenta que gera automaticamente o código necessário para carregar as funções OpenGL.

No presente projeto, o GLAD é configurado para OpenGL 4.6 Core Profile e é inicializado em `SDLWindow::Create` através da função:

- `gladLoadGLLoader((GLADloadproc)SDL_GL_GetProcAddress)`

2.4.5 Síntese e Implicações

OpenGL 4.6 Core Profile é uma escolha adequada para o projeto, pelas seguintes razões:

- **Maturidade** e documentação abundante;
- **Portabilidade** (Windows, Linux, macOS via *deprecated*);
- **Desempenho** adequado para jogos 2D;
- **Pipeline moderno** baseado em shaders;
- **Integração** natural com SDL3.

2.5 O Padrão Entidades-Componentes-Sistemas (ECS)

2.5.1 Definição

O **Entity-Component-System** (ECS) é um padrão arquitetural que emerge da convergência de duas correntes: a **programação orientada a dados** (*Data-Oriented Design*) e a **composição sobre a herança**. Foi popularizado por projetos como o *Dungeon Siege* (2002) e, mais recentemente, por motores como o Unity e o Bevy. A sua premissa central é radicalmente diferente da programação orientada a objetos tradicional: **em vez de agrupar dados e comportamento numa mesma estrutura (o objeto), o ECS separa-os em três elementos distintos.**

- **Entidade:** identificador único - tipicamente um número inteiro - que não contém dados nem comportamento. A sua única função é servir de "etiqueta" à qual componentes podem ser associados. Uma entidade sozinha é, por si só, vazia: representa "algo que existe no mundo" mas sem qualquer propriedade ou capacidade. O código abaixo mostra a parte inicial da função `CreateSpriteEntity`, que mostra como é atribuído o ID a um sprite e no fim retorna o valor da variável `entity`;

```
entt::entity ECSScene::CreateSpriteEntity(const Vector3D& position, const Vector3D& scale,
std::shared_ptr<Texture2D> texture){
    auto entity = m_Registry.create();
```

Figura 3- Criação de um Sprite

- **Componente:** estrutura de dados pura, sem lógica. Não tem métodos (ou tem apenas construtores simples), não herda de outras classes, e não conhece outras entidades. Um componente descreve uma propriedade ou capacidade que pode ser associada a uma entidade. No código abaixo, o componente `Transform` ao associar a uma entidade (`Player`, `Enemy`), define a sua posição e rotação no mundo, e tamanho (`Scale`);

```
struct Transform{
    Vector3D Position{0.0f, 0.0f, 0.0f}; //World position in 3D space (Z is depth in 2D)
    Vector3D Rotation{0.0f, 0.0f, 0.0f}; //Euler angles in radians (Tait-Bryan)
    Vector3D Scale{1.0f, 1.0f, 1.0f}; //Scale factors along each axis
};
```

Figura 4- Componente Transform

- **Sistema:** função ou classe que processa entidades que possuem um conjunto específico de componentes. Os sistemas contêm toda a lógica do motor: renderização, física, IA, colisões, etc. Enquanto os componentes são dados puros, os sistemas são o "motor" que os transforma em comportamento. No presente projeto, os sistemas estão organizados na pasta engine/systems/ e são implementados como classes com métodos static, atualmente foram implementados os sistemas de inimigos e renderização

2.5.2 Vantagens do ECS

Comparado com a herança profunda (comum em motores antigos), o ECS oferece:

- **Flexibilidade:** novas combinações de comportamentos são obtidas pela simples adição de componentes;
- **Testabilidade:** sistemas são funções puras, facilmente testáveis;
- **Desacoplamento:** sistemas não conhecem outros sistemas diretamente;
- **Reutilização:** componentes e sistemas podem ser reutilizados em diferentes jogos.

2.5.3 EnTT — Detalhes

EnTT é uma biblioteca *header-only* que implementa um ECS com as seguintes características:

- **Registo (entt::registry):** contentor central que armazena entidades e componentes;
- **Views:** filtros que permitem iterar sobre entidades que possuem determinados componentes;
- **Grupos:** otimizações de *views* para *queries* frequentes;

- **Pools:** contentores especializados por tipo de componente;
- **Signal/sink:** sistema de callbacks para eventos de criação/destruição.

2.5.4 Síntese e Implicações

O ECS é um padrão adequado para o presente projeto porque:

- Facilita a adição de novos tipos de entidades sem alterar código existente;
- Promove a separação entre dados (componentes) e lógica (sistemas);
- Oferece desempenho adequado mesmo com muitas entidades;
- É amplamente utilizado na indústria moderna (Godot 4.x, Bevy, Unity DOTS).

2.6 Padrões de Design Aplicáveis

2.6.1 Game Loop

O **Game Loop** é o padrão central de qualquer motor de jogo. Consiste num ciclo infinito que, a cada iteração:

- Processa eventos de input;
- Atualiza o estado do jogo;
- Renderiza a cena.

O presente projeto implementa o *Game Loop* em `Application::Run`, com separação clara entre `ProcessEvents`, `Update` e `Render`.

2.6.2 Singleton

O padrão Singleton é utilizado em vários subsistemas (`EventBus`, `ResourceManager`, `SoundManager`, `GamepadManager`) para garantir uma única instância global. Embora seja

frequentemente criticado por dificultar testes, é adequado para subsistemas verdadeiramente globais.

2.6.3 Observer / Event Bus

O padrão Observer é implementado através do EventBus, que permite desacoplar emissores e recetores de eventos. Este padrão é fundamental para manter o baixo acoplamento entre módulos.

2.6.4 State / Scene Stack

O padrão State é implementado através do SceneManager, que mantém uma pilha de cenas e delega Update e Render para a cena ativa.

2.6.5 Component / ECS

O padrão **Component** é implementado pelo ECS. Em vez de herança profunda, os comportamentos são compostos por componentes, permitindo uma flexibilidade muito maior.

2.6.6 Síntese e Implicações

Os padrões adotados no projeto são complementares e reforçam a arquitetura modular:

- **Game Loop** organiza o tempo;
- **Singleton** gere subsistemas globais;
- **Observer** desacopla eventos;
- **State** gere cenas;
- **Component/ECS** compõe entidades.

2.7 Sistemas de Áudio em Motores de Jogo

2.7.1 Desafios do Áudio

O áudio em jogos apresenta desafios específicos:

- **Latência:** sons devem ser reproduzidos com latência mínima (< 50 ms);
- **Conversão de formatos:** ficheiros WAV podem estar em 44.1 kHz/16-bit/estéreo, mas o dispositivo pode estar em 48 kHz;
- **Mistura:** múltiplos sons simultâneos requerem mistura;
- **Streaming:** música longa não deve ser carregada inteiramente em memória.

2.7.2 Abordagem do Presente Projeto

O presente projeto adota uma abordagem simples mas robusta:

- **Carregamento completo** do ficheiro WAV em memória (adequado para efeitos curtos);
- **Conversão automática** de formatos via `SDL_AudioStream`;
- **Cache de sons** via `SoundManager`;
- **Volume controlável** por efeito.

Esta abordagem é adequada para efeitos sonoros curtos, mas não para música longa (que exigiria *streaming*).

2.8 Sistemas de Eventos em Motores de Jogo

2.8.1 Necessidade de Desacoplamento

Em motores de jogo, é comum que múltiplos sistemas precisem de reagir a eventos (ex: quando o jogador salta, o áudio toca um som, a UI atualiza, e a física aplica um impulso). Sem um sistema de eventos, estes sistemas teriam de estar fortemente acoplados, dificultando a manutenção.

2.8.2 Abordagem do Presente Projeto

O `EventBus` implementa um sistema de eventos baseado em:

- **Subscrição por tipo** (`std::type_index`): cada tipo de evento tem a sua lista de *callbacks*;
- **Despacho síncrono**: os *callbacks* são invocados no momento do despacho;
- **Singleton global**: acessível em qualquer parte do motor.

Este sistema é simples, eficiente e adequado para o escopo do projeto.

2.9 Comparação de Abordagens e Síntese Final

A Tabela 2 compara as principais abordagens discutidas neste capítulo.

Tabela 2 - Comparação SDL2 vs SDL3

Dimensão	Abordagem A	Abordagem B	Escolha do Projeto
Plataforma	SDL2	SDL3	SDL3 (API moderna)
Renderização	OpenGL Legacy	OpenGL Core	OpenGL 4.6 Core
Arquitetura	Herança	ECS	ECS (EnTT)
Eventos	Acoplado	EventBus	EventBus
Cenas	Única	Stack	Stack
Áudio	Direto	Streams	Streams (SDL3)

3. Metodologia

Este capítulo descreve a abordagem metodológica adotada ao longo do desenvolvimento do projeto. Começa por classificar o tipo de projeto, justifica a escolha da metodologia, descreve o plano de avaliação, enumera as ferramentas utilizadas e documenta as principais dificuldades encontradas e as respetivas soluções.

3.1 Abordagem Metodológica

3.1.1 Contexto e desafios

O contexto de desenvolvimento apresentou três características determinantes que condicionaram a escolha metodológica:

- **Natureza individual:** o projeto foi desenvolvido por um único autor, sem equipa;
- **Incerteza técnica elevada:** a integração de múltiplas bibliotecas heterogéneas (SDL3, OpenGL, GLAD, EnTT, SDL_ttf) num único sistema era um desafio novo, sem experiência prévia consolidada;
- **Prazo fixo:** aproximadamente dois meses de desenvolvimento efetivo, com marcos intermédios definidos pelo calendário académico.

Estas características foram determinantes para a adoção de uma metodologia mais pragmática.

3.1.2 Metodologia adotada

Não foi adotada uma metodologia formal única. Em vez disso, optou-se por uma abordagem **pragmática e adaptativa**, inspirada em princípios de **desenvolvimento iterativo e incremental** e em **desenvolvimento exploratório com controlo de versões**, ajustada à natureza individual do projeto e à incerteza técnica associada à integração de múltiplas bibliotecas heterogéneas.

- O Kanban foi considerado, mas acabou por ser rejeitado por três razões:
 - Pressupõe **fluxo contínuo** de trabalho, incompatível com a natureza irregular do trabalho académico (picos de intensidade entre avaliações);
 - O **limite de WIP** perde sentido num projeto individual, onde existe um único recurso;
 - As **métricas de fluxo** (*lead time, throughput, cycle time*) não têm valor estatístico com um único executor.
- O Scrum foi igualmente considerado e rejeitado:
 - Pressupõe uma **equipa** com papéis definidos (Product Owner, Scrum Master, Development Team);
 - Exige **eventos formais** (*Daily Scrum, Sprint Review, Retrospective*) que, automaticamente torna-se irrelevante num projeto a *solo*;
 - Impõe **sprints de duração fixa**, incompatíveis com a irregularidade do trabalho académico.

3.1.3 Pilares da Abordagem Adotada

3.1.3.1 Desenvolvimento Iterativo e Incremental

O desenvolvimento foi organizado em **incrementos com espaço temporal variável** (4-7 dias). Cada incremento tinha:

- **Objetivo definido:** uma funcionalidade concreta a implementar;
- **Demo funcional:** a funcionalidade era demonstrada na cena de demonstração;

3.1.3.2 Desenvolvimento Exploratório com Git

Em vez de criar protótipos isolados em ambiente separado, optou-se por um **desenvolvimento exploratório diretamente no repositório principal**, utilizando o Git como ferramenta central de suporte. O processo seguia cinco passos:

1. **Implementar:** escrever o código da funcionalidade no próprio projeto;
2. **Testar localmente:** compilar e executar a cena de demonstração;
3. **Corrigir:** se falhasse, iterar sobre o código até funcionar;
4. **Commitar:** após validação, fazer commit com mensagem descritiva;
5. **Envio para o GitHub:** sincronizar o repositório remoto.

Esta abordagem tem vantagens e desvantagens face à prototipagem clássica:

- **Vantagens:**
 - **Contexto real:** o código é testado no ambiente final, não num protótipo isolado;
 - **Sem duplicação:** não há código descartável a manter paralelamente;
 - **Rastreabilidade:** cada commit documenta uma tentativa bem-sucedida;
 - **Segurança:** o Git permite reverter para o último estado funcional se algo falhar.
- **Desvantagens:**
 - **Menor isolamento:** um erro pode quebrar temporariamente o projeto;
 - **Histórico ruidoso:** commits frequentes de tentativas falhadas podem poluir o histórico (mitigado com mensagens descritivas);
 - **Sem ambiente controlado:** não há um "laboratório" separado para experimentar sem risco.

Um dos casos, foi a migração dos namespaces que foi desenvolvida, testada, código quebrado foi corrigido e só depois da reconstrução do projeto e a compilação ter sucesso, foi feito o seu commit e lançamento para o repositório. No caso de implementação de código incompleto mas em que a sua funcionalidade não quebra o funcionamento do resto do projeto, em caso de necessidade, concretamente trocar de computador para continuar o projeto, foi também lançado os commit, com comentário acerca da nova funcionalidade ainda não funcional.

3.1.3.3 Adaptação contínua

A adaptação contínua foi o terceiro pilar. Semanalmente, o plano era revisto com base em dois critérios:

- **Dificuldades encontradas:** tarefas bloqueadas eram repriorizadas ou desdobradas;
- **Conhecimento adquirido:** novas técnicas descobertas eram incorporadas nos incrementos seguintes.

Um dos casos reais, em que inicialmente, o plano previa uma cena de menu completa com transições funcionais. Ao aproximar-se o prazo de entrega, tornou-se claro que o tempo restante não era suficiente para completar o menu e o relatório. Optou-se por:

- **Reduzir o escopo do menu:** manter apenas as opções essenciais (Play, Exit);
- **Priorizar a documentação:** garantir que Doxygen e o relatório ficassem completos;
- **Documentar a limitação:** reconhecer no relatório que o menu ficou parcialmente implementado.

3.2 Plano de Avaliação

A avaliação do motor foi realizada de forma contínua, através de várias técnicas complementares:

3.2.1 Testes de Integração Manuais

Após cada incremento, a cena de demonstração era executada e validada manualmente. Os aspetos verificados incluíam:

- Movimento do jogador (setas, WASD, espaço);
- Colisões com plataformas e inimigos;
- Renderização correta de sprites e texto;
- Reprodução correta de sons;
- Estabilidade (ausência de *crashes*).

3.2.2 Análise de Logs

O sistema de logging permitiu identificar:

- Erros de carregamento de recursos (texturas, sons, fontes);
- Falhas de compilação de shaders;
- Problemas de inicialização de subsistemas (áudio, gamepad);
- Estados anómalos (entidades não geradas corretamente, câmara mal posicionada).

3.2.3 Métricas de Desempenho

O FpsCounter forneceu dados sobre a fluidez da renderização, com logs periódicos (a cada segundo) contendo:

- Frames por segundo;
- Delta time (tempo entre cada frame renderizado);
- Tempo decorrido.

3.2.4 Inspeção Visual

Verificação visual da cena, incluindo:

- Correta renderização de sprites (sem artefactos);
- Correta renderização de *tiling*.

3.2.5 Análise Estática

O compilador foi configurado com avisos rigorosos (-Wall -Wextra -Wpedantic), e o código foi periodicamente revisto para eliminar avisos.

3.3 Ferramentas Utilizadas

A tabela abaixo resume as ferramentas utilizadas no desenvolvimento.

Tabela 3 - Versionamento das ferramentas utilizadas

Ferramenta	Versão	Propósito
CMake	4.4.2	Sistema de build
Ninja	1.13	Gerador de build
Clang	22.1.8	Compilador
SDL3	3.4.0	Plataforma, input, áudio
SDL3_ttf	3.3.0	Renderização de texto
OpenGL	4.6	Renderização
GLAD	0.1.36	Loader OpenGL
EnTT	4.0.0	ECS
stb_image	2.30	Carregamento de texturas
Doxygen	1.18	Documentação
Git	2.55	Controlo de versões
Kate	26.08.0	Editor
CachyOS (Arch Linux) / Windows 11	—	Sistema operativo

3.4 Cronograma e Fases

O desenvolvimento foi organizado em nove fases lógicas, definidas com base no histórico de commits do repositório Git. Cada fase corresponde a um conjunto coeso de funcionalidades ou a um marco significativo no progresso do motor:

- **Fase 1 - Fundações:** Configuração inicial do projeto: repositório Git, estrutura de pastas, CMakeLists.txt base, integração do SDL3 como submódulo e adição das bibliotecas GLAD. Definição das primeiras decisões arquiteturais (OpenGL Core vs Compatibility, estrutura de diretórios).
- **Fase 2 – Núcleo do Motor:** Criação da camada SDL, inicialização da janela e do runtime SDL3, refatoração do *main loop* para a classe Application, implementação do sistema de tempo (Time), configuração inicial do logging com `std::format` e adição das macros de assertão.
- **Fase 3 – Sistema de eventos:** Implementação das classes de eventos (Event, EventBus, KeyEvent, MouseEvent), do EventDispatcher e do SDL_EventTranslator. Expansão iterativa do sistema de eventos e refatoração do código associado.
- **Fase 4 – Camada Gráfica:** Implementação dos shaders, dos buffers (VertexBuffer, VertexArray, IndexBuffer), do *quad* base, da função de renderização e do carregamento de texturas com `stb_image`. Conversão do GLAD para código-fonte e integração com o sistema de build.
- **Fase 5 – ECS e Estruturação:** Implementação da Camera2D, do *input* de teclado e rato, da integração da biblioteca EnTT para o ECS, do escalonamento de sprites e do ResourceManager. Refatoração geral do código e expansão das capacidades de logging e eventos.
- **Fase 6 – Jogo (Demonstração):** Implementação do GameWorld, do *logging throttling*, ajustes de cena e criação das primeiras cenas de demonstração 2D. Adição de colisões com plataformas, ajuste da física de salto e implementação da cena de demonstração com câmara fixa.

- **Fase 7 – Áudio e Gamepad:** Implementação do sistema de áudio (SoundEffect, SoundManager), com testes de sons de salto e aterragem. Adição do suporte a gamepads.
- **Fase 8 – Inimigos e mecânicas de Jogo:** Implementação dos inimigos com patrulha e física, expansão do *roadmap* do projeto e integração das mecânicas de *stomping*.
- **Fase 9 – Menu, Tilling e Refinamento:** Cópia automática de DLLs no Windows, documentação com Doxygen, expansão das capacidades de *tileScale* para repetição de texturas, ajustes no redimensionamento de janela, implementação do menu funcional e limpeza final do código.

3.5 Dificuldades Encontradas

Durante o desenvolvimento, surgiram várias dificuldades técnicas significativas. A Tabela 4 resume as principais.

Tabela 4 - Dificuldades encontradas

#	Dificuldade	Impacto	Solução
1	DLLs não eram copiadas	Médio	Usar <code>add_custom_command</code> com <i>generator expressions</i>
2	Namespaces inconsistentes	Baixo	Uniformizar todos os componentes em <code>Components</code>
3	Texto invertido/desfocado	Baixo	Ajustar filtragem (com <code>GL_NEAREST</code>) e <i>flip</i> correto
4	Áudio com formato errado	Médio	Usar <code>SDL_GetAudioDeviceFormat</code>
5	Cliques do rato no menu	Baixo	Conversão de coordenadas de ecrã → mundo
6	Erros de <i>reorder</i> em construtores	Baixo	Ordem correta de inicialização

3.5.1 Detalhe das Dificuldades Mais Relevantes

Dificuldade 1 - Cópia automática de DLLs no Windows. No Windows, o executável depende de SDL3.dll e SDL3_ttf.dll. O CMake não as copia automaticamente, pelo que era necessário um passo de pós-build. A solução foi usar `add_custom_command` que resolve o caminho real da DLL e copia para a pasta do executável em tempo de build.

Dificuldade 2 - Namespaces inconsistentes. Inicialmente, alguns componentes estavam no *global namespace* (Enemy, Patrol, TileScale) e outros em namespace Components (Transform, PhysicsBody, SpriteRenderer). Isto causou erros de compilação quando se tentou usar Enemy sem o prefixo. A solução foi uniformizar todos em Components, exigindo a atualização de todos os usos no código.

Dificuldade 3 - Texto invertido/desfocado. O texto renderizado com SDL_ttf aparecia invertido verticalmente e desfocado. A primeira parte resolveu-se com o *flip* da superfície SDL antes de a carregar para OpenGL. A segunda parte foi resolvida ajustando a filtragem: GL_NEAREST para magnificação (texto mais nítido) e GL_LINEAR para minificação.

Dificuldade 4 - Formato do dispositivo de áudio. O SDL3 pode abrir o dispositivo com uma especificação diferente da pedida (ex: 48 kHz em vez de 44.1 kHz). A solução foi usar `SDL_GetAudioDeviceFormat` para obter a especificação real e usá-la na criação das *audio streams*.

Dificuldade 5 - Cliques do rato no menu. A deteção de cliques do rato exigiu conversão de coordenadas de ecrã (em píxeis) para coordenadas de mundo (na projeção ortográfica do menu). A solução envolveu uma transformação linear simples.

4. Especificação de Requisitos

4.1 Introdução

Este capítulo especifica os requisitos funcionais e não-funcionais do motor de jogo. Os requisitos foram definidos no início do projeto e refinados ao longo do desenvolvimento, refletindo a natureza iterativa da metodologia adotada.

4.2 Requisitos Funcionais

Os requisitos funcionais descrevem o que o motor deve fazer. A Tabela 5 abaixo lista os RF relevantes.

Tabela 5 - Requisitos funcionais

ID	Requisito	Descrição	Prioridade	Estado
RF01	Janela	Criar janela com contexto OpenGL	Alta	OK
RF02	Renderização 2D	Renderizar sprites 2D com textura	Alta	OK
RF03	Tiling	Suportar repetição de textura	Baixa	OK
RF04	Transparência	Suportar alpha blending	Alta	OK
RF05	Áudio	Reproduzir efeitos sonoros WAV	Média	OK
RF06	Volume	Controlar volume por efeito	Baixa	OK

ID	Requisito	Descrição	Prioridade	Estado
RF07	Input	Suportar teclado, rato, gamepad	Alta	OK
RF08	ECS	Gerir entidades via ECS	Alta	OK
RF09	Cenas	Suportar múltiplas cenas	Média	OK
RF10	Transições	Transitar entre cenas	Média	OK
RF11	Texto	Renderizar texto com SDL_ttf	Média	OK
RF12	Menu	Menu com opções navegáveis	Média	Parcial
RF13	Física	Colisões AABB	Alta	OK
RF14	Patrulha	IA simples de patrulha	Média	OK
RF15	Stomping	Mecânica de saltar em cima	Baixa	OK

4.2.1 Detalhe dos Requisitos Funcionais

RF01 - Janela: O motor deve criar uma janela com um contexto OpenGL 4.6 Core Profile, com suporte a redimensionamento e modo *fullscreen*.

RF02 - Renderização 2D: O motor deve renderizar *sprites* 2D texturizados, com suporte a transformações (posição, rotação, escala).

RF03 - Tiling: O motor deve permitir a repetição de texturas (útil para chão, paredes, etc.).

RF04 - Transparência: O motor deve suportar *alpha blending* para permitir sprites com transparência.

RF05 - Áudio: O motor deve reproduzir efeitos sonoros em formato WAV, com conversão automática de formatos.

RF06 - Volume: O motor deve permitir controlar o volume de cada efeito sonoro individualmente.

RF07 - Input: O motor deve suportar input de teclado, rato e gamepad, com deteção de *pressed, held e released*.

RF08 - ECS: O motor deve usar o padrão ECS para gestão de entidades, com componentes e sistemas desacoplados.

RF09 - Cenas: O motor deve permitir múltiplas cenas, cada uma com o seu próprio estado.

RF10 - Transições: O motor deve permitir transições entre cenas (ex: menu → jogo → menu).

RF11 - Texto: O motor deve renderizar texto com SDL_ttf, com suporte a diferentes tamanhos e cores.

RF12 - Menu: O motor deve ter um menu principal com opções (Play, Options, Exit), navegável por teclado, rato e gamepad.

RF13 - Física: O motor deve suportar colisões AABB entre entidades.

RF14 - Patrulha: O motor deve suportar IA simples para patrulha horizontal para inimigos.

RF15 - Stomping: O motor deve suportar a mecânica de saltar em cima de inimigos para os destruir.

4.3 Requisitos Não-Funcionais

Os requisitos não-funcionais descrevem como o motor deve comportar-se.

Tabela 6 - Requisitos não funcionais

ID	Requisito	Métrica	Estado
RNF01	Desempenho	≥ 60 FPS com vsync	OK
RNF02	Modularidade	Separação clara em módulos	OK
RNF03	Portabilidade	Compilação em Windows e Linux	OK
RNF04	Documentação	Código documentado com Doxygen	OK
RNF05	Manutenibilidade	Baixo acoplamento	OK
RNF06	Extensibilidade	Adicionar novas funcionalidades	OK
RNF07	Robustez	Sem <i>crashes</i> em uso normal	OK
RNF08	Logging	Sistema de logging funcional	OK

4.4 Casos de Uso

4.4.1 Caso de Uso 1 — Iniciar o Motor

Ator: Utilizador

Pré-condições: Sistema com SDL3 e OpenGL instalados

Fluxo principal: 1. Utilizador executa Game.exe; 2. Motor inicializa logging, SDL3, SDL_ttf, janela, OpenGL, ECS, cenas; 3. Motor carrega a cena inicial (menu ou demo); 4. Motor entra no *game loop*.

Pós-condições: Janela aberta com a cena inicial a renderizar.

4.4.2 Caso de Uso 2 — Movimentar o Jogador

Ator: Jogador

Pré-condições: Demo2DFixedScene ativa

Fluxo principal: 1. Jogador pressiona seta direita; 2. Motor deteta input no Update do PlayerController; 3. Motor atualiza a posição do jogador; 4. Motor resolve colisões com plataformas; 5. Motor renderiza o jogador na nova posição.

Pós-condições: Jogador moveu-se para a direita.

4.4.3 Caso de Uso 3 — Saltar em Cima de Inimigo

Ator: Jogador

Pré-condições: Jogador e inimigo em colisão vertical

Fluxo principal: 1. Jogador salta e cai sobre o inimigo; 2. CheckEnemyCollisions deteta a colisão; 3. Verifica se o jogador está a cair ($velocity.y < 0$) e acima do inimigo; 4. Destroi o inimigo; 5. Aplica impulso vertical ao jogador; 6. Reproduz som (se implementado).

Pós-condições: Inimigo destruído, jogador saltou.

4.5 Restrições e Pressupostos

4.5.1 Restrições

4.5.1.1 Restrições Temporais

O projeto foi desenvolvido no âmbito de uma unidade curricular com duração de um semestre letivo. O tempo disponível foi dividido em três períodos distintos:

- **Planeamento (2–3 meses):** período durante o qual foram exploradas diferentes ideias de projeto (um modelo LLM treinado de raiz e uma rede de servidores com Active Directory, DNS e DHCP), antes de se optar definitivamente pelo motor de jogo. Este período foi marcado pela necessidade de avaliar o custo de oportunidade de cada alternativa.
- **Desenvolvimento (aproximadamente 2 meses):** período efetivo de implementação, durante o qual o motor foi construído iterativamente.
- **Documentação e relatório (2–3 semanas finais):** período dedicado à geração da documentação Doxygen e à redação do presente relatório.

A restrição temporal teve implicações diretas no escopo do projeto: várias funcionalidades inicialmente previstas (menu completo, animações de sprites, sistema de partículas, editor de níveis) foram conscientemente adiadas ou reduzidas, em favor de garantir que o núcleo do motor ficasse funcional e documentado.

4.5.1.2 Restrições de Escopo

Conscientemente, o escopo do motor foi limitado a **2D**. Esta decisão foi tomada por três razões:

- **Complexidade do 3D:** um motor 3D exigiria um pipeline de renderização muito mais complexo (iluminação, câmaras perspetivas, gestão de profundidade), incompatível com o prazo disponível;
- **Adequação ao objetivo pedagógico:** o objetivo principal era compreender os princípios arquiteturais de um motor, não a complexidade gráfica. Um motor 2D bem estruturado cumpre esse objetivo;
- **Foco na arquitetura:** ao limitar a dimensão gráfica, foi possível dedicar mais tempo à arquitetura (ECS, eventos, cenas, recursos), que era o foco central do projeto.

Adicionalmente, o motor foi limitado a OpenGL, excluindo suporte a Vulkan ou Direct3D 12, e a ficheiros WAV para áudio, excluindo formatos comprimidos como OGG ou MP3.

No entanto, é importante sublinhar que estas limitações são de escopo atual, não de arquitetura. O motor foi desenhado desde o início com a extensibilidade em mente: a abstração da camada gráfica (GraphicsContext, Shader, Texture2D) e a separação entre o renderer (Renderer) e a lógica de jogo (ECSScene) permitem que, no futuro, sejam adicionados:

- **Suporte a 3D:** a estrutura de Transform já inclui coordenadas Z, e a câmara (Camera2D) poderia ser complementada por uma Camera3D sem alterar o resto do motor;
- **Suporte a Vulkan ou Direct3D 12:** a camada de abstração gráfica isola o resto do código das especificidades da API. Substituir OpenGL por outra API implicaria reescrever apenas a pasta graphics/, mantendo intactos o ECS, os sistemas, as cenas e a lógica de jogo;
- **Suporte a OGG/MP3:** o SoundManager está preparado para receber novos formatos, bastando adicionar um novo carregador.

Esta decisão de design - limitar o escopo atual, mas preparar a arquitetura para o futuro - é consistente com os princípios de modularidade e baixo acoplamento que orientaram todo o projeto

4.5.2 Pressupostos

4.5.2.1 Pressupostos de Ambiente de Execução

O motor assume que o sistema onde é executado cumpre os seguintes requisitos:

- **OpenGL 4.6 Core Profile:** a GPU e os drivers devem suportar esta versão. Sem ela, a inicialização do contexto OpenGL falha e o motor termina;
- **Runtime Visual C++ (Windows):** em Windows, o executável depende do runtime Visual C++ (msvcp140.dll, vcruntime140.dll). Este runtime está presente na maioria dos sistemas modernos, mas pode ter de ser instalado manualmente;

- **SDL3 e SDL3_ttf:** as DLLs do SDL3 e SDL3_ttf devem estar presentes na mesma pasta do executável (o CMake copia-as automaticamente em tempo de build);
- **Pasta assets:** o motor assume que a pasta assets/ (contendo subpastas textures/, sounds/ e fonts/) está presente no diretório de trabalho. Sem ela, os recursos não são carregados;
- **Pasta logs/:** o motor cria automaticamente a pasta logs/ no arranque, se não existir. Assume que o diretório tem permissões de escrita.

4.5.2.2 Pressupostos de Utilização

O motor foi concebido para ser utilizado por um programador com conhecimentos básicos de:

- **C++ moderno:** compreensão de *smart pointers*, referências, templates e *move semantics*;
- **Conceitos de ECS:** compreensão do padrão Entidades-Componentes-Sistemas;
- **Programação gráfica:** noções básicas de shaders, buffers e texturas;
- **CMake:** capacidade de configurar e compilar o projeto.

Não é assumido que o utilizador tenha experiência prévia em OpenGL ou SDL3 — o motor abstrai essas complexidades.

4.5.2.3 Pressupostos Técnicos

O motor assume vários pressupostos técnicos internos:

- **Uma única janela:** o motor atualmente suporta apenas uma janela por instância da aplicação. Não há suporte para múltiplas janelas em simultâneo até nova implementação;
- **Uma única thread de renderização:** toda a renderização corre na thread principal. Não há suporte para renderização assíncrona ou multithreaded;

- **Uma única instância do motor:** o Application foi concebido para ser instanciado apenas uma vez. Não há suporte para múltiplas instâncias;
- **Assets pré-carregados ou carregados em runtime:** o motor assume que os assets existem no disco e são acessíveis. Não há suporte para *streaming* de assets durante o jogo (exceto áudio de curta duração, que é carregado inteiramente em memória);
- **Dispositivo de áudio disponível:** o SoundEffect assume que existe um dispositivo de áudio funcional. Se não existir, o motor continua a funcionar (com logs de aviso), mas sem reprodução de som.

4.5.2.4 Pressupostos de Manutenção

O motor foi desenhado com a manutenção em mente, mas assume:

- **Documentação atualizada:** o código está documentado com Doxygen. Se for modificado, espera-se que a documentação seja atualizada em conformidade;
- **Compatibilidade com versões fixas:** o motor foi testado com versões específicas das bibliotecas utilizadas. Versões mais recentes podem introduzir incompatibilidades;
- **Estrutura de pastas estável:** o motor assume que a estrutura de pastas (engine/, assets/, vendor/) permanece inalterada.

5. Implementação

5.1 Introdução ao Capítulo

Este capítulo é o núcleo do relatório. Descreve em detalhe a arquitetura do motor de jogo, módulo a módulo, com excertos de código, decisões de design e justificações. O objetivo não é descrever exaustivamente cada linha de código, mas sim explicar o desenho da solução e as razões que presidiram às escolhas técnicas.

Sempre que relevante, são referidas as dificuldades encontradas e as respetivas soluções, com o objetivo de documentar não apenas o resultado final, mas também o percurso. Esta transparência é consistente com a abordagem metodológica adotada (Capítulo 3) e com a reflexão crítica do Capítulo 6.

O capítulo está organizado em dezassete secções, correspondentes aos módulos principais do motor e a aspetos transversais:

- **5.2 Arquitetura Global** – visão de conjunto e princípios arquiteturais
- **5.3 Módulo core** – orquestração, janela, tempo, input, controladores
- **5.4 Módulo graphics** – abstração OpenGL (shaders, buffers, texturas)
- **5.5 Módulo renderer** – API de desenho de quads
- **5.6 Módulo scene** – cenas, câmara, mundo, ECS
- **5.7 Módulo systems** – sistemas ECS (renderização, inimigos)
- **5.8 Módulo components** – componentes ECS
- **5.9 Módulo audio** – som
- **5.10 Módulo events** – sistema de eventos
- **5.11 Módulo resources** – gestão de recursos

- **5.12 Módulo diagnostics** – logging e asserts
- **5.13 Módulo math** – vetores e matrizes
- **5.14 Cena de Demonstração** – Demo2DFixedScene
- **5.15 Menu Principal** – MenuScene
- **5.16 CMake e Build System** – configuração de build
- **5.17 Tratamento de Erros e Robustez** – estratégia de falha controlada

Para cada módulo, descreve-se: (1) a responsabilidade, (2) as classes principais, (3) as decisões de design e respetiva justificação, (4) os fluxos de execução relevantes, e (5) as dificuldades encontradas.

5.2 Arquitetura Global

5.2.1 Visão Geral

A arquitetura do motor segue uma organização em camadas, com os seguintes módulos principais:

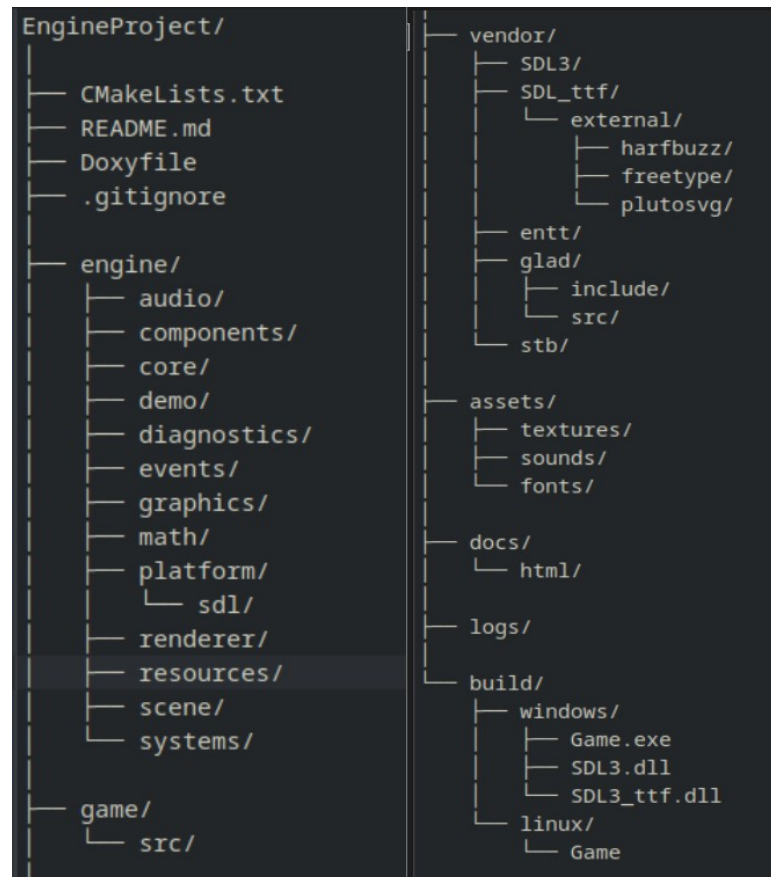


Figura 5 - Estrutura do projeto

A separação entre engine/ e game/ é deliberada: o motor é genérico e reutilizável, enquanto game/src/main.cpp contém apenas a lógica específica da aplicação (criação da cena inicial, configuração da janela). Esta separação permite que o motor seja reutilizado noutros projetos sem alterações.

5.2.2 Princípios Arquiteturais

A arquitetura foi desenhada segundo seis princípios fundamentais, que orientaram todas as decisões técnicas ao longo do desenvolvimento:

- **Separação de responsabilidades;** Cada módulo tem uma responsabilidade única e bem definida. O módulo audio/ trata exclusivamente de som; o módulo graphics/ trata exclusivamente de renderização; o módulo events/ trata exclusivamente de eventos. Não há sobreposição de responsabilidades.
- **Baixo acoplamento:** Os módulos comunicam através de interfaces mínimas. O EventBus é o principal mecanismo de comunicação desacoplada: os sistemas não se conhecem diretamente, apenas subscrevem a eventos. O ResourceManager é outro exemplo: os sistemas pedem texturas sem saber como são carregadas.
- **Alta coesão:** Cada módulo agrupa funcionalidades relacionadas. O módulo graphics/ contém todas as classes relacionadas com OpenGL; o módulo math/ contém todas as classes matemáticas. Esta coesão facilita a navegação no código e a compreensão da arquitetura.
- **Substituibilidade:** As bibliotecas de terceiros são encapsuladas atrás de abstrações. O SDL3 é encapsulado por SDLWindow e Input; o OpenGL é encapsulado por GraphicsContext e Shader; o EnTT é encapsulado por ECSScene. Substituir qualquer destas bibliotecas implicaria alterações apenas no módulo correspondente, não em todo o motor.
- **Testabilidade:** Os módulos podem ser testados isoladamente. Embora não tenham sido implementados testes unitários (limitação reconhecida no Capítulo 6), a arquitetura foi desenhada com a testabilidade em mente: classes com

responsabilidades bem definidas, dependências injetadas por referência, ausência de estado global excessivo.

- **Extensibilidade.** O motor foi desenhado para crescer. A adição de novos componentes ECS, novos sistemas, novas cenas ou novas funcionalidades não exige alterações profundas na arquitetura. A integração de uma biblioteca de física (ex: Box2D) implicaria alterações apenas no módulo `systems/`.

5.2.3 Fluxo de Execução

- O fluxo de execução do motor segue a estrutura clássica de um *game loop*:
- **main** cria uma instância de `Application` e chama `Initialize()`;
- **Application::Initialize()** inicializa, por ordem: logging, SDL, SDL_ttf, janela, OpenGL, câmara, mundo, gamepad, FPS counter, e cena inicial;
- **Application::Run()** entra no *game loop*, que se repete enquanto o motor estiver em execução:
 - **Time::Update()** — atualiza o delta time;
 - **ProcessEvents()** — processa eventos SDL (input, janela, gamepad);
 - **Update()** — atualiza a cena ativa (`SceneManager::Update`);
 - **Input::Update()** — atualiza o estado do input e reinicia os deltas;
 - **Render()** — renderiza a cena ativa (`SceneManager::Render`);
- **Application::Shutdown()** liberta todos os recursos, na ordem inversa da inicialização.

A ordem das operações é importante: o input é atualizado no final do loop (após o Update), para que os deltas de rato e scroll sejam reiniciados antes do próximo frame. Esta decisão evita que um mesmo evento seja processado duas vezes.

5.2.4 Dependências

As dependências principais são:

- core depende de graphics, events, scene, audio, resources, diagnostics;
- scene depende de graphics, components, systems, math;
- systems depende de components, graphics, renderer, math;
- graphics depende de math, diagnostics;
- Todos os módulos dependem de diagnostics (para logging) e math (para vetores e matrizes).

Não há dependências circulares. O módulo events é independente de todos os outros (apenas usa math para vetores em eventos específicos), o que permite que seja reutilizado em qualquer contexto.

5.3 Módulo core

5.3.1 Descrição

O módulo core contém as classes centrais do motor:

- **Application:** orquestrador principal (inicialização, loop, shutdown);
- **Engine:** gestão de janela e SDL e estado de execução;
- **Time:** gestão de tempo (delta time, elapsed time);
- **Input:** gestão de input (teclado, rato);
- **GamepadManager:** gestão de gamepads;

- **PlayerController:** controlador do jogador;
- **CameraController:** controlador da câmara;
- **FpsCounter:** contador de FPS.

5.3.2 Application

Application é o orquestrador principal do motor. As suas responsabilidades incluem:

- Inicializar todos os subsistemas, numa ordem específica;
- Executar o *game loop*;
- Gerir o SceneManager;
- Subscrever eventos globais (ex: WindowCloseEvent);
- Libertar recursos no Shutdown.

A ordem de inicialização não é arbitrária. O logging é inicializado primeiro (para registar erros das inicializações seguintes); o SDL é inicializado em segundo (sem ele, não há janela nem contexto OpenGL); o SDL_ttf é inicializado em terceiro (para renderização de texto); a janela e suas subscrições em eventos é criada em quarto; o contexto OpenGL é criado em quinto (requer a janela); o GraphicsContext é inicializado em sexto (requer o contexto OpenGL); a câmara e o mundo são criados em sétimo; os controladores em oitavo.

O código simplificado do Initialize é:

```
bool Application::Initialize(){
    Log::Initialize("logs/");
    if(SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO) < 0) return false;
    if(TTF_Init() != 0) Log::Error("TTF init failed");
    if(!m_Engine.Initialize("Game Window", 1920, 1080, false)) return false;

    //Subscrever eventos
    EventBus::GetInstance().Subscribe<WindowCloseEvent>([this](Event& e){
        m_Engine.SetRunning(false);
    });

    //Inicializar OpenGL
    m_Graphics = std::make_unique<GraphicsContext>();
    if(!m_Graphics->Initialize()) return false;

    //Câmara
    int winWidth, winHeight;
    SDL_GetWindowSize(m_Engine.GetWindow().GetNativeWindow(), &winWidth, &winHeight);
    float aspect = static_cast<float>(winWidth) / winHeight;
    float height = 5.0f;
    float width = height * aspect;
    m_Camera = std::make_unique<Camera2D>(-width, width, -height, height);

    //Mundo
    m_World = std::make_unique<GameWorld>(-20.0f, 20.0f, -10.0f, 10.0f);

    //Gamepad
    GamepadManager::GetInstance().Initialize();

    //FPS
    m_FpsCounter = std::make_unique<FpsCounter>();

    Log::Info("Application initialized successfully.");
    return true;
}
```

Figura 6 - Função Initialize()

Optou-se por usar `std::unique_ptr` para a maioria dos subsistemas (`m_Graphics`, `m_Camera`, `m_World`, `m_FpsCounter`), o que garante que os recursos são libertados automaticamente quando o `Application` é destruído. O `SceneManager` é um membro direto (não um `unique_ptr`), pois é um objeto leve que não gere recursos externos. O `Engine` é um membro direto pela mesma razão.

O Application não é um singleton, mas o comentário na documentação Doxygen alerta para não criar mais do que uma instância. Esta decisão foi tomada para facilitar testes (um singleton seria mais difícil de substituir) e para deixar explícito que o Application é o ponto de entrada, não um serviço global.

O *game loop* é:

```
void Application::Run(){
    while(m_Engine.IsRunning()){
        Time::Update();
        ProcessEvents();
        Update();
        Input::Update();
        Render();
    }
}
```

Figura 7 - Função Run()

A ausência de *fixed timestep* é uma limitação reconhecida: o Update usa o delta time diretamente, o que pode causar variações no comportamento físico em sistemas com FPS instável. Para o escopo atual (jogo 2D simples), esta abordagem é aceitável.

5.3.3 Engine

Engine é responsável por:

- Criar a janela SDL (delegando para SDLWindow);
- Gerir o estado m_Running;
- Fornecer acesso à janela (GetWindow).

A classe `Engine` é deliberadamente minimalista: não gere eventos, não gere input, não renderiza. Apenas cria e destrói a janela, e mantém o estado de execução. Esta separação permite que o `Application` controle o *game loop* sem depender de detalhes da janela.

5.3.4 Time

`Time` é uma classe estática que fornece `DeltaTime()` e `ElapsedTime()`. Usa `SDL_GetPerformanceCounter` para medições de alta precisão.

Optou-se por uma classe estática em vez de uma instância porque o tempo é uma grandeza global: todos os sistemas precisam de saber o delta time, e não faz sentido ter múltiplas instâncias. A variável `previousCounter` é `static` dentro da função, o que garante que persiste entre chamadas.

5.3.5 Input

`Input` é uma classe estática que gere o estado do teclado, rato e janela. Oferece métodos como:

- **`IsKeyPressed(SDL_Scancode)`** – tecla premida neste frame (transição);
- **`IsKeyHeld(SDL_Scancode)`** – tecla mantida;
- **`IsKeyReleased(SDL_Scancode)`** – tecla libertada neste frame;
- **`IsMouseButtonPressed(uint8_t)`** – botão do rato premido;
- **`GetMousePosition()`** – posição do rato;
- **`GetScrollDelta()`** – delta de scroll;
- **`WasWindowResized()`** – se a janela foi redimensionada.

A classe usa *arrays* estáticos (`s_KeyState`, `s_KeyStatePrevious`) em vez de contentores dinâmicos, o que é mais eficiente. O estado atual é obtido via `SDL_GetKeyboardState`, que devolve um *array* de `bool` com o estado de todas as teclas.

A dupla atualização do estado do teclado (via eventos e via `SDL_GetKeyboardState`) foi uma fonte inicial de confusão. A solução adotada foi usar apenas `SDL_GetKeyboardState` no `Update`, ignorando os eventos de teclado no `ProcessEvent`. Isto garante que o estado é sempre consistente com o estado real do teclado.

5.3.6 GamepadManager

`GamepadManager` é um singleton que gere gamepads. Suporta:

- Detecção de conexão/desconexão (`SDL_EVENT_GAMEPAD_ADDED/REMOVED`);
- Leitura de botões e eixos.

Optou-se por um singleton porque existe, no máximo, um conjunto de gamepads ligados ao sistema, e todos os sistemas que precisam de input de gamepad devem aceder ao mesmo estado. O `GamepadManager` usa `std::unordered_map<int, SDL_Gamepad*>` para mapear índices de jogador (0–3) para gamepads.

5.3.7 PlayerController

`PlayerController` gere o jogador. A sua lógica inclui:

- Leitura de input (teclado + gamepad);
- Movimento horizontal;
- Salto com altura variável;
- Gravidade;
- Detecção de *grounded*;
- Morte e respawn.

O código do Update é extenso mas pode ser resumido nos seguintes passos:

```
void PlayerController::Update(){
    if(m_IsDead) return;

    //Input
    Vector2D keyboardDir(0.0f, 0.0f);
    if(Input::IsKeyHeld(SDL_SCANCODE_LEFT)) keyboardDir.x -= 1.0f;
    if(Input::IsKeyHeld(SDL_SCANCODE_RIGHT)) keyboardDir.x += 1.0f;

    //Gamepad
    Vector2D gamepadDir(0.0f, 0.0f);
    if(GamepadManager::GetInstance().IsConnected(0)){
        gamepadDir.x = GamepadManager::GetInstance().GetAxis(0, SDL_GAMEPAD_AXIS_LEFTX);
    }

    //Aplicar movimento
    //...

    //Salto
    if(wantToJump && m_IsGrounded){
        m_IsJumping = true;
        m_Velocity.y = 12.0f;
        SoundManager::GetInstance().PlaySound("assets/sounds/jump.wav");
    }

    //Física vertical
    //...
}
```

Figura 8 - Função Update() da classe PlayerController

O PlayerController não usa o sistema ECS para o jogador; é um controlador dedicado. Esta decisão foi tomada porque o jogador tem lógica específica (salto variável, morte, respawn) que não se encaixa bem no modelo ECS genérico. O jogador tem componentes ECS (Transform, SpriteRenderer), mas a lógica é gerida pelo PlayerController, não por um sistema ECS.

5.3.8 CameraController

CameraController gere a câmara em dois modos:

- **Follow:** segue o jogador com interpolação linear;
- **Manual:** controlada por WASD.

O modo *follow* é usado na Demo2DScene (câmara dinâmica); o modo manual é usado para depuração.

Decisão de design. A interpolação linear (lerp) foi escolhida em vez de movimento direto para suavizar o seguimento da câmara. Um fator de 0.05 significa que a câmara percorre 5% da distância ao alvo em cada frame, o que resulta num movimento suave e natural.

O *clamping* da câmara aos limites do mundo evita que o jogador veja o "vazio" fora do mundo. A verificação `if(minX < maxX)` foi adicionada para evitar comportamento indefinido quando o mundo é mais pequeno que as margens.

5.3.9 FpsCounter

FpsCounter calcula a média de FPS a cada segundo e regista-a no log. Usa um temporizador interno e um contador de frames.

5.4 Módulo graphics

5.4.1 Descrição

O módulo graphics abstrai o OpenGL, oferecendo classes para:

- **Shader:** compilação e uso de shaders;
- **VertexBuffer:** gestão de VBO;
- **VertexArray:** gestão de VAO;
- **IndexBuffer:** gestão de IBO;
- **Texture2D:** carregamento e uso de texturas;
- **Font:** carregamento de fontes TTF;
- **GraphicsContext:** gestão centralizada.

5.4.2 GraphicsContext

GraphicsContext é o ponto central do módulo. As suas responsabilidades incluem:

- Criar os buffers (VBO, VAO, IBO) para um *quad* unitário;
- Compilar os shaders (principal e de linhas);
- Configurar os atributos de vértice;
- Ativar o *alpha blending*;
- Fornecer acesso aos recursos (GetShader, GetVertexArray, etc.).

O GraphicsContext gere um único *quad* unitário, que é reutilizado para todos os sprites. Cada sprite é desenhado com uma transformação diferente (matriz de modelo), mas a geometria é sempre a mesma. Esta abordagem reduz o consumo de memória e simplifica o código.

O *quad* unitário tem vértices em $(-0.5, -0.5)$ a $(0.5, 0.5)$, com coordenadas de textura em $(0, 0)$ a $(1, 1)$. A escala é aplicada pela matriz de modelo, não pelos vértices.

5.4.3 Shader

Shader compila e linka shaders GLSL. Oferece métodos para definir *uniforms*:

- SetUniformMat4(name, matrix);
- SetUniformFloat(name, value);
- SetUniformVec2(name, x, y);
- SetUniformFloat3(name, x, y, z).

O shader principal é compilado em `GraphicsContext::CompileShaders`:

```
//Vertex shader
#version 460 core
layout(location = 0) in vec3 a_Position;
layout(location = 1) in vec2 a_TexCoord;
uniform mat4 u_Model;
uniform mat4 u_View;
uniform mat4 u_Projection;
uniform vec2 u_TileScale;
out vec2 v_TexCoord;
void main(){
    gl_Position = u_Projection * u_View * u_Model * vec4(a_Position, 1.0);
    v_TexCoord = a_TexCoord * u_TileScale;
}
//Fragment shader
#version 460 core
in vec2 v_TexCoord;
out vec4 FragColor;
uniform sampler2D u_Texture;
void main(){
    FragColor = texture(u_Texture, v_TexCoord);
}
```

Figura 9 - Compilação de Shaders

O *vertex shader* aplica as matrizes de projeção, vista e modelo, e passa as coordenadas de textura (escaladas por `u_TileScale`) para o *fragment shader*. O *fragment shader* amostra a textura e devolve a cor. Esta é a abordagem mais simples e eficiente para renderização 2D.

A ordem das multiplicações (`u_Projection * u_View * u_Model`) é importante: em OpenGL, a matriz de projeção é aplicada por último, mas a multiplicação é feita da direita para a esquerda (a matriz de modelo é aplicada primeiro ao vértice).

5.4.4 Texture2D

`Texture2D` carrega texturas de ficheiros (via `stb_image`) ou de `SDL_Surface` (para texto). Suporta:

- GL_REPEAT para *tiling*;
- Mipmaps;
- Filtragem configurável.

O carregamento via `stb_image` usa `stbi_set_flip_vertically_on_load(true)`, o que inverte a imagem verticalmente. Isto é necessário porque o SDL e o OpenGL têm convenções diferentes para a origem das coordenadas: o SDL usa o canto superior esquerdo, o OpenGL usa o canto inferior esquerdo.

O carregamento via `SDL_Surface` (usado para texto) também aplica o *flip* manualmente, copiando as linhas da superfície para um *buffer* invertido. Esta duplicação deve-se ao facto de `SDL_ConvertSurface` não suportar o *flip* diretamente.

5.4.5 Font

Font usa `SDL_ttf` para renderizar texto para `SDL_Surface`, que pode ser convertido em `Texture2D`. O método `RenderText` usa `TTF_RenderText_Blended`, que produz texto com *anti-aliasing*.

Optou-se por `TTF_RenderText_Blended` em vez de `TTF_RenderText_Solid` porque o texto *blended* tem melhor qualidade visual, apesar de ser ligeiramente mais lento. Para um menu, a diferença de desempenho é irrelevante.

5.5 Módulo renderer

5.5.1 Descrição

O módulo `renderer` contém a classe `Renderer`, que oferece uma API estática para desenhar *quads* (retângulos) texturizados ou não texturizados. Esta classe é a camada de abstração entre o `RenderSystem` (que processa entidades ECS) e o OpenGL (que executa os *draw calls*).

5.5.2 Implementação

A classe `Renderer` oferece dois métodos:

- **`DrawQuad(shader, va, ib)`** – desenha um *quad* sem textura (apenas geometria);
- **`DrawTexturedQuad(shader, va, ib, texture, model, view, projection, tileX, tileY)`** – desenha um *quad* texturizado.

O método `DrawTexturedQuad` é o principal:

```
void Renderer::DrawTexturedQuad(Shader& shader, VertexArray& va, IndexBuffer& ib,
                                Texture2D& texture, const Matrix4& model,
                                const Matrix4& view, const Matrix4& projection,
                                float tileX, float tileY){
    shader.Bind();
    texture.Bind();
    shader.SetUniformMat4("u_Model", model);
    shader.SetUniformMat4("u_View", view);
    shader.SetUniformMat4("u_Projection", projection);
    shader.SetUniformVec2("u_TileScale", tileX, tileY);
    va.Bind();
    ib.Bind();
    glDrawElements(GL_TRIANGLES, ib.GetCount(), GL_UNSIGNED_INT, nullptr);
}
```

Figura 10 - Função `DrawTexturedQuad()`

5.5.3 Decisões de Design

A classe `Renderer` foi desenhada com três princípios em mente:

- **Simplicidade** – todos os métodos são estáticos, pois o `Renderer` não mantém estado próprio;
- **Abstração mínima** – o `Renderer` não gere buffers nem shaders; apenas os usa. A gestão é responsabilidade do `GraphicsContext`;

- **Independência do ECS** – o `Renderer` não conhece o `ECS`; recebe matrizes e texturas como parâmetros. Isto permite que seja reutilizado fora do contexto `ECS`.

A desvantagem da API estática é que dificulta testes unitários (não se pode substituir por um *mock*). No entanto, dado que o `Renderer` apenas delega para `OpenGL`, os testes seriam limitados de qualquer forma.

5.5.4 Fluxo de Renderização

O percurso de um `sprite` desde o `ECS` até ao ecrã envolve quatro camadas:

1. **`ECSScene::Render`** itera sobre o registo `ECS` e delega para o `RenderSystem`;
2. **`RenderSystem::Render`** filtra entidades com `Transform` e `SpriteRenderer`, constrói a matriz de modelo e chama o `Renderer`;
3. **`Renderer::DrawTexturedQuad`** liga o `shader`, a `textura`, o `VAO` e o `IBO`, define os *uniforms* e executa o *draw call*;
4. **`OpenGL`** processa os vértices, aplica os `shaders` e escreve no *framebuffer*.

5.6 Módulo scene

5.6.1 Descrição

O módulo `scene` contém:

- **`Scene`**: classe base abstrata;
- **`SceneManager`**: gestor de cenas;
- **`Camera2D`**: câmara ortográfica;
- **`GameWorld`**: mundo do jogo com *clamping*;
- **`ECSScene`**: cena `ECS`;
- **`MenuScene`**: menu principal;

- **GameObject:** classe legada (não usada).

5.6.2 Scene e SceneManager

Scene define a interface que todas as cenas devem implementar :

```
class Scene{
public:
    virtual ~Scene() = default;
    virtual void OnEnter() = 0;
    virtual void OnExit() = 0;
    virtual void Update(float deltaTime) = 0;
    virtual void Render() = 0;
    virtual std::string GetName() const = 0;
};
```

Figura 11 - Classe Scene

A interface é minimalista: OnEnter e OnExit para inicialização e limpeza; Update e Render para o loop; GetName para identificação (usada pelo Application para decidir se deve atualizar a câmara em caso de redimensionamento).

O SceneManager mantém uma pilha de cenas (`std::stack<std::unique_ptr<Scene>>`) e delega Update e Render para a cena ativa. As operações suportadas são:

- **SetScene** – substitui a cena atual;
- **PushScene** – empilha uma nova cena (a anterior fica pausada);
- **PopScene** – desempilha a cena atual (volta à anterior).

A pilha permite transições suaves entre cenas (ex: menu → jogo → menu) e pausas (ex: jogo → pausa → jogo). A cena anterior não é destruída quando uma nova é empilhada, o que preserva o seu estado.

5.6.3 Camera2D

Camera2D gere uma projeção ortográfica com posição, rotação e zoom. A matriz de vista é calculada no Update:

```
void Camera2D::Update(){
    if(!m_IsDirty) return;
    Matrix4 view = Matrix4::Translation(Vector3D(-m_Position.x, -m_Position.y, -m_Position.z));
    if(m_Rotation != 0.0f){
        Matrix4 rot = Matrix4::RotationZ(-m_Rotation);
        view = rot * view;
    }
    m_ViewMatrix = view;
    m_IsDirty = false;
}
```

Figura 12 - Função Update() da classe Camera2D

A câmara usa uma *dirty flag* (`m_IsDirty`) para evitar recalcular a matriz de vista em cada frame. A matriz só é recalculada quando a posição ou rotação mudam. Esta otimização é relevante em cenas com câmara estática (como a `Demo2DFixedScene`), onde a matriz de vista nunca muda.

A projeção ortográfica é recalculada sempre que o zoom ou o tamanho da janela mudam. A fórmula é:

```
1 left = -m_Width * (1.0f / m_Zoom)
2 right = m_Width * (1.0f / m_Zoom)
3 bottom = -m_Height * (1.0f / m_Zoom)
4 top = m_Height * (1.0f / m_Zoom)
```

Um zoom maior que 1.0 reduz o volume de visualização (aproxima); um zoom menor que 1.0 aumenta o volume (afasta).

5.6.4 GameWorld

GameWorld define os limites do mundo e fornece ClampPosition. Também renderiza as bordas com um *line shader*.

As bordas do mundo são renderizadas como linhas vermelhas, o que ajuda na depuração e na visualização dos limites. Em produção, estas linhas seriam desativadas ou substituídas por elementos visuais (ex: paredes texturizadas).

O ClampPosition é usado pelo PlayerController para impedir que o jogador saia do mundo:

```
1 Vector3D GameWorld::ClampPosition(const Vector3D& position) const{
2     Vector3D clamped = position;
3     clamped.x = std::max(m_MinX, std::min(m_MaxX, clamped.x));
4     clamped.y = std::max(m_MinY, std::min(m_MaxY, clamped.y));
5     return clamped;
6 }
```

5.6.5 ECSScene

ECSScene encapsula um entt::registry, oferecendo CreateSpriteEntity e Render.

O ECSScene não é uma subclasse de Scene; é uma classe auxiliar usada pelas cenas concretas (Demo2DFixedScene, Demo2DScene). Esta separação permite que o ECS seja reutilizado em diferentes contextos, não apenas em cenas.

5.7 Módulo systems

5.7.1 Descrição

O módulo systems contém:

- RenderSystem: renderiza entidades com Transform + SpriteRenderer;

- EnemySystem: atualiza inimigos com patrulha, gravidade e colisões.

5.7.2 RenderSystem

O RenderSystem itera sobre todas as entidades que têm Transform e SpriteRenderer, constrói a matriz de modelo e chama o Renderer:

```
static void Render(entt::registry& registry, Shader& shader,
                  const Matrix4& view, const Matrix4& projection,
                  VertexArray& va, IndexBuffer& ib){
    auto view_entities = registry.view<Components::Transform, Components::SpriteRenderer>();
    for(auto [entity, transform, sprite] : view_entities.each()){
        if(!sprite.Texture) continue;
        Matrix4 model = Matrix4::Translation(transform.Position);
        model = model * Matrix4::RotationZ(transform.Rotation.z);
        model = model * Matrix4::Scale(transform.Scale);

        float tileX = 1.0f, tileY = 1.0f;
        if(registry.all_of<Components::TileScale>(entity)){
            const auto& ts = registry.get<Components::TileScale>(entity);
            tileX = ts.ScaleX;
            tileY = ts.ScaleY;
        }

        Renderer::DrawTexturedQuad(shader, va, ib, *sprite.Texture,
                                   model, view, projection, tileX, tileY);
    }
}
```

Figura 13 - Método Render() da classe RenderSystem

A ordem das transformações é translação → rotação → escala, o que é a convenção padrão em gráficos 2D. Isto significa que a rotação é aplicada em torno do centro do sprite, e a escala é aplicada após a rotação.

A verificação `if(!sprite.Texture)` `continue` evita *crashes* quando uma entidade tem SpriteRenderer mas a textura não foi carregada. Esta é uma aplicação da estratégia de falha controlada.

5.7.3 EnemySystem

O EnemySystem processa inimigos com Transform, Enemy, Patrol e PhysicsBody. A lógica inclui:

- **Patrulha horizontal** – move o inimigo entre StartPosition e EndPosition;
- **Gravidade** – aplica gravidade à velocidade vertical;
- **Colisões com plataformas** – resolve colisões AABB;
- **Clamping ao chão** – impede que o inimigo caia abaixo do mundo;
- **Atualização do estado grounded** – regista se o inimigo está no chão.

O código principal é:

```
auto view = registry.view<Components::Transform, Components::Enemy,
Components::Patrol, Components::PhysicsBody>();
for(auto [entity, transform, enemy, patrol, physics] : view.each()){
    if(!enemy.IsActive) continue;

    //Patrulha
    float speed = patrol.Speed * deltaTime;
    if(patrol.MovingRight){
        transform.Position.x += speed;
        if(transform.Position.x >= patrol.EndPosition.x) patrol.MovingRight = false;
    } else{
        transform.Position.x -= speed;
        if(transform.Position.x <= patrol.StartPosition.x) patrol.MovingRight = true;
    }

    //Gravidade
    physics.Velocity.y += -9.8f * physics.GravityScale * deltaTime;
    transform.Position.y += physics.Velocity.y * deltaTime;

    //Colisões
    //...
}
```

Figura 14 - Método Update() da classe EnemySystem

Optou-se por uma abordagem simples de patrulha (mover até ao fim, inverter direção) em vez de uma IA mais complexa. Para o escopo atual, é suficiente. Em projetos futuros, poderia ser substituída por uma máquina de estados ou por *pathfinding*.

5.8 Módulo components

5.8.1 Descrição

O módulo components contém as estruturas de dados ECS:

- **Transform:** posição, rotação, escala;
- **SpriteRenderer:** textura;
- **PhysicsBody:** velocidade, *grounded*, *gravity scale*;
- **Enemy:** tag de inimigo;
- **Patrol:** patrulha (start, end, speed, direction);
- **TileScale:** escala de *tiling*.

Todos os componentes estão em namespace Components.

5.8.2 Implementação

Transform:

```
1 struct Transform{  
2     Vector3D Position{0.0f, 0.0f, 0.0f};  
3     Vector3D Rotation{0.0f, 0.0f, 0.0f};  
4     Vector3D Scale{1.0f, 1.0f, 1.0f};  
5 };
```

SpriteRenderer:

```
1 struct SpriteRenderer{  
2     std::shared_ptr<Texture2D> Texture;
```

```
3 };
```

Enemy:

```
1 struct Enemy{  
2     bool IsActive = true;  
3 };
```

Patrol:

```
1 struct Patrol{  
2     Vector3D StartPosition;  
3     Vector3D EndPosition;  
4     float Speed = 2.0f;  
5     bool MovingRight = true;  
6 };
```

Os componentes são *structs* simples, sem métodos (exceto construtores). Esta é a filosofia do ECS: os componentes são dados puros, a lógica está nos sistemas. A ausência de métodos facilita a serialização e a cópia.

5.9 Módulo audio

5.9.1 Descrição

O módulo audio contém:

- **SoundEffect:** carrega um ficheiro WAV e reproduz-o;
- **SoundManager:** singleton que faz cache de efeitos.

5.9.2 SoundEffect

SoundEffect::Load carrega o ficheiro WAV via `SDL_LoadWAV_IO`. SoundEffect::Play cria uma `SDL_AudioStream` e converte o formato do ficheiro para o formato real do dispositivo (obtido via `SDL_GetAudioDeviceFormat`).

O SoundEffect carrega o ficheiro WAV inteiro para memória, em vez de fazer *streaming*. Esta abordagem é adequada para efeitos curtos (saltos, colisões), mas não para música longa. A conversão de formato é feita automaticamente pelo SDL3, o que simplifica o código.

5.9.3 SoundManager

SoundManager faz cache de `shared_ptr<SoundEffect>`, evitando carregar o mesmo som duas vezes.

5.9.4 Dificuldade Encontrada

O SDL3 pode abrir o dispositivo com uma especificação diferente da pedida (ex: 48 kHz em vez de 44.1 kHz). A solução foi usar `SDL_GetAudioDeviceFormat` para obter a especificação real:

```
1 if(!SDL_GetAudioDeviceFormat(s_AudioDevice, &m_DeviceSpec, nullptr)){  
2     Log::Error(std::format("Failed to get actual device format: {}", SDL_GetError()));  
3     SDL_CloseAudioDevice(s_AudioDevice);  
4     return false;  
5 }
```

5.10 Módulo events

5.10.1 Descrição

O módulo events contém:

- **Event:** classe base;

- **EventBus:** singleton de subscrição e despacho;
- **SDLTranslator:** converte eventos SDL em eventos do motor;
- Eventos específicos (KeyEvent, MouseEvent, PlayerDiedEvent, etc.).

5.10.2 EventBus

O EventBus implementa um sistema de eventos baseado em subscrição por tipo (`std::type_index`) e despacho síncrono:

```
template<typename T>
void Subscribe(const Callback& callback){
    std::type_index typeIndex = std::type_index(typeid(T));
    m_Subscribers[typeIndex].push_back(callback);
}

void Dispatch(Event& event){
    std::type_index typeIndex = std::type_index(typeid(event));
    auto it = m_Subscribers.find(typeIndex);
    if(it != m_Subscribers.end()){
        for(auto& callback : it->second){
            callback(event);
        }
    }
}
```

Figura 15 - Captura de eventos e seu handling

Optou-se por subscrição por tipo em vez de subscrição por instância porque é mais eficiente: cada tipo de evento tem uma lista de *callbacks*, e o despacho é uma pesquisa num `unordered_map`. A alternativa (subscriver a eventos específicos) exigiria armazenar identificadores de eventos, o que seria mais complexo.

Sem o EventBus, o PlayerController teria de conhecer diretamente o Input (ou o SDL), criando acoplamento. Com o EventBus, o PlayerController apenas conhece a interface Event

e o tipo `KeyEvent` – não sabe de onde vêm os eventos, nem quantos outros sistemas estão a ouvir.

5.10.3 Fluxo de um Evento

O percurso de um evento de teclado, desde o SDL até à lógica de jogo, ilustra o funcionamento do sistema:

1. **SDL captura o evento** (`SDL_EVENT_KEY_DOWN`);
2. **`Application::ProcessEvents`** chama `SDLEventTranslator::TranslateAndDispatch`;
3. **`SDLEventTranslator::DispatchKeyEvent`** cria um `KeyEvent` e chama `EventBus::Dispatch`;
4. **`EventBus::Dispatch`** itera sobre os *callbacks* subscritos para `KeyEvent`;
5. **O `PlayerController`** (que subscreveu no construtor) recebe o evento e reage.

5.11 Módulo resources

5.11.1 Descrição

O módulo `resources` contém a classe `ResourceManager`, um singleton que faz cache de texturas e shaders, evitando carregamentos duplicados.

O `ResourceManager` não tem política de despejo (*eviction*): uma vez carregada, uma textura permanece em cache até o motor ser encerrado. Para o escopo atual, isto é aceitável. Em projetos maiores, seria necessário implementar uma política de LRU (*Least Recently Used*) ou carregamento por nível.

5.11.2 ResourceManager

```
1 std::shared_ptr<Texture2D> ResourceManager::LoadTexture(const std::string& filepath){  
2     //Check if texture is already cached  
3     auto it = m_Textures.find(filepath);
```

```

4  if(it != m_Textures.end()) return it->second;

5

6  //Load texture from disk

7  auto texture = std::make_shared<Texture2D>();

8  if(!texture->Load(filepath)){

9      Log::Error(std::format("Failed to load texture: {}", filepath));

10     return nullptr;

11 }

12

13 //Add texture to cache

14 m_Textures[filepath] = texture;

15 Log::Info(std::format("Texture loaded and cached: {}", filepath));

16 return texture;

17 }

```

O ResourceManager usa `std::shared_ptr` para as texturas, o que permite que múltiplas entidades partilhem a mesma textura sem duplicar memória. Quando a última referência é destruída, a textura é libertada automaticamente.

A cache é um `std::unordered_map<std::string, std::shared_ptr<Texture2D>>`, indexado pelo caminho do ficheiro. A pesquisa é $O(1)$ em média, o que torna o *cache hit* praticamente instantâneo.

5.12 Módulo diagnostics

5.12.1 Descrição

O módulo diagnostics contém:

- **Log:** sistema de logging com níveis e ficheiro;
- **Assert:** macro `ENGINE_ASSERT`.

Log suporta *throttling*, permitindo evitar spam de mensagens repetidas.

5.12.2 Log

O Log oferece quatro níveis (Info, Warning, Error, Debug) e suporta *throttling* para evitar spam:

```
1 void Log::InfoThrottled(const std::string& message, const std::string& key, float intervalSeconds){
2     auto now = std::chrono::steady_clock::now();
3     float currentTime = std::chrono::duration<float>(now.time_since_epoch()).count();
4
5     auto it = s_LastLogTime.find(key);
6     if(it == s_LastLogTime.end() || (currentTime - it->second) >= intervalSeconds){
7         s_LastLogTime[key] = currentTime;
8         Write(LogLevel::Info, message);
9     }
10 }
```

O *throttling* é útil para mensagens que seriam registadas em cada frame (ex: posição do jogador). Sem ele, o log cresceria rapidamente e tornaria-se ilegível. A chave (key) permite que diferentes mensagens tenham *throttling* independente.

5.12.3 Assert

A macro `ENGINE_ASSERT` verifica uma condição e, se falhar, regista um erro e aborta o programa:

```
1 #define ENGINE_ASSERT(condition, message) \
2 do{ \
3     if(!(condition)){ \
4         Log::Error(std::format("Assertion Failed: {}\\nFile: {}\\n Line: {}", message, \
5             __FILE__, __LINE__)); \
6         std::abort(); \
7     } \
8 } while(false)
```

A macro é usada apenas em situações de "não deveria acontecer" (ex: textura nula após carregamento bem-sucedido).

5.13 Módulo math

5.13.1 Descrição

O módulo math contém:

- **Vector2D, Vector3D:** vetores com operações básicas;
- **Matrix4:** matriz 4x4 com translação, rotação, escala e projeção ortográfica.

5.13.2 Matrix4

A `Matrix4` armazena os dados em *column-major order* (formato OpenGL) e oferece métodos estáticos para criar matrizes:

- **Identity()** – matriz identidade;
- **Translation(position)** – matriz de translação;
- **Scale(scale)** – matriz de escala;
- **RotationZ(angle)** – matriz de rotação em torno do eixo Z;
- **Orthographic(left, right, bottom, top, near, far)** – matriz de projeção ortográfica.

A multiplicação de matrizes é implementada manualmente:

```
1 Matrix4 Matrix4::operator*(const Matrix4& other) const {
2     Matrix4 result;
3
4     for(int row = 0; row < 4; ++row){
5         for(int col = 0; col < 4; ++col){
6             result.m_Data[col * 4 + row] =
7                 m_Data[0 * 4 + row] * other.m_Data[col * 4 + 0] +
8                 m_Data[1 * 4 + row] * other.m_Data[col * 4 + 1] +
9                 m_Data[2 * 4 + row] * other.m_Data[col * 4 + 2] +
10                m_Data[3 * 4 + row] * other.m_Data[col * 4 + 3];
11         }
12     }
```

```
13     return result;  
14 }
```

Optou-se por implementar as matrizes manualmente em vez de usar uma biblioteca (ex: GLM) para compreender os fundamentos da matemática gráfica. O *column-major order* é a convenção do OpenGL, o que evita conversões.

5.14 Cena de Demonstração (Demo2DFixedScene)

5.14.1 Descrição

A Demo2DFixedScene implementa uma pequena plataforma com:

- Câmara fixa que mostra todo o mundo;
- Jogador controlável;
- Plataformas estáticas e flutuantes;
- Inimigos com patrulha;
- Colisões AABB;
- Mecânica de *stomping*;
- Respawn de inimigos.

5.14.2 Estrutura

A cena tem as seguintes entidades:

- 1 jogador;
- 7 plataformas (incluindo paredes);
- 3 inimigos.

O jogador é controlado pelo PlayerController, não por um sistema ECS. Os inimigos são controlados pelo EnemySystem.

5.14.3 SetupScene

O SetupScene cria todas as entidades:

```
void Demo2DFixedScene::SetupScene(){
    m_ECSScene = std::make_unique<ECSScene>(m_Shader, m_VertexArray, m_IndexBuffer);
    m_World = std::make_unique<GameWorld>(-20.0f, 20.0f, -10.0f, 10.0f);

    //Câmara fixa no centro
    m_Camera.SetProjection(m_World->GetMinX(), m_World->GetMaxX(),
                           m_World->GetMinY(), m_World->GetMaxY());
    m_Camera.SetPosition(Vector3D(0.0f, 0.0f, 0.0f));
    m_Camera.Update();

    //Carregar texturas
    auto playerTex = ResourceManager::GetInstance().LoadTexture("assets/textures/player.png");
    auto tileTex = ResourceManager::GetInstance().LoadTexture("assets/textures/tile.png");

    //Criar jogador
    m_PlayerEntity = m_ECSScene->CreateSpriteEntity(
        Vector3D(0.0f, 0.0f, 0.0f), Vector3D(3.0f, 3.0f, 1.0f), playerTex);

    //Criar plataformas...

    //Criar inimigos
    SpawnEnemies();

    //Controladores
    m_PlayerController = std::make_unique<PlayerController>(
        m_ECSScene->GetRegistry(), m_PlayerEntity, *m_World);
    m_PlayerController->SetSpeed(4.0f);

    m_FpsCounter = std::make_unique<FpsCounter>();
}
```

Figura 16 - Função SetupScene() para criação da cena

5.14.4 Fluxo de Atualização

O Update da cena segue esta ordem:

1. **m_PlayerController->Update()** – processa input, aplica movimento e gravidade;

2. **ResolveCollisions()** – resolve colisões jogador-plataforma e atualiza o estado *grounded*;
3. **EnemySystem::Update()** – atualiza patrulha e gravidade dos inimigos;
4. **CheckEnemyCollisions()** – verifica colisões jogador-inimigo e aplica *stomping* ou morte;
5. **m_FpsCounter->Update()** – atualiza métricas de desempenho.

O jogador é atualizado antes das colisões, para que a posição final seja corrigida. Os inimigos são atualizados depois, para que a colisão seja verificada com as posições finais de ambos. Se a ordem fosse invertida, o jogador poderia colidir com um inimigo que já se moveu, mas a posição do jogador ainda não foi corrigida.

5.14.5 CheckEnemyCollisions

A função `CheckEnemyCollisions` implementa a mecânica de *stomping*:

```
void Demo2DFixedScene::CheckEnemyCollisions(){
    auto& registry = m_ECSScene->GetRegistry();
    auto& playerTransform = registry.get<Components::Transform>(m_PlayerEntity);
    Vector3D playerHalf = playerTransform.Scale * 0.5f;
    float playerBottom = playerTransform.Position.y - playerHalf.y;

    auto enemyView = registry.view<Components::Transform, Components::Enemy,
    Components::PhysicsBody>();

    for(auto [entity, transform, enemy, physics] : enemyView.each()){
        if(!enemy.IsActive) continue;

        Vector3D enemyHalf = transform.Scale * 0.5f;
        float dx = std::abs(playerTransform.Position.x - transform.Position.x);
        float dy = std::abs(playerTransform.Position.y - transform.Position.y);
        bool colliding = (dx < (playerHalf.x + enemyHalf.x)) &&
            (dy < (playerHalf.y + enemyHalf.y));
        if(!colliding) continue;

        float enemyTop = transform.Position.y + enemyHalf.y;
        bool isAbove = (playerBottom < enemyTop + 0.1f) && (playerBottom > enemyTop - 0.2f);
        bool isFalling = (m_PlayerController->GetVelocity().y < 0.0f);

        if(isFalling && isAbove){
            DestroyEnemy(entity);
            m_PlayerController->GetVelocity().y = 5.0f;
            Log::Info("Enemy stomped!");
        } else{
            m_PlayerController->Die("Killed by enemy");
            RespawnEnemies();
            break;
        }
    }
}
```

Figura 17 - Detecção de colisões com a função `CheckEnemyCollisions()`

A detecção de *stomping* usa três condições: (1) o jogador está a cair ($\text{velocity.y} < 0$), (2) o fundo do jogador está próximo do topo do inimigo (*isAbove*), e (3) há colisão AABB. Esta combinação evita falsos positivos (ex: jogador a subir colide com inimigo de lado). As margens $+0.1f$ e $-0.2f$ foram ajustadas empiricamente para tornar a mecânica mais tolerante.

5.15 Menu Principal (MenuScene)

5.15.1 Descrição

O MenuScene apresenta três opções: *Play*, *Options*, *Exit*. Cada opção tem duas texturas: uma normal (branca) e uma selecionada (amarela). A seleção é feita por:

- Setas/WASD (navegação);
- Enter/Espaço (confirmação);
- Rato (clique direto).

5.15.2 Criação de Texto

O texto é renderizado com `SDL_ttf` para uma `SDL_Surface`, que é convertida em `Texture2D`:

```
std::shared_ptr<Texture2D> MenuScene::CreateTextTexture(const std::string& text,
                                                         const SDL_Color& color){
    SDL_Surface* surface = m_Font->RenderText(text, color);
    auto texture = std::make_shared<Texture2D>();
    texture->LoadFromSurface(surface);

    glBindTexture(GL_TEXTURE_2D, texture->GetRenderID());
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
    glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);

    SDL_DestroySurface(surface);
    return texture;
}
```

Figura 18 - Renderização de texto

Optou-se por GL_NEAREST para magnificação (texto mais nítido) e GL_LINEAR para minificação (evita serrilhado). Esta combinação foi escolhida após testes com diferentes filtragens: GL_LINEAR em ambas resultava em texto desfocado; GL_NEAREST em ambas resultava em texto pixelizado.

5.15.3 Clique do Rato

O clique do rato é detetado no Update, convertendo coordenadas do ecrã para coordenadas de mundo:

```
if(Input::IsMouseButtonPressed(1)){
    Vector2D mousePos = Input::GetMousePosition();
    float worldX = (mousePos.x / winWidth) * 20.0f - 10.0f;
    float worldY = -(mousePos.y / winHeight) * 20.0f + 10.0f;

    for(size_t i = 0; i < m_Options.size(); ++i){
        //Verificar se o clique está dentro da bounding box
        //...
    }
}
```

Figura 19 - Captura do clique do rato

A conversão usa a projeção ortográfica do menu (de -10 a 10 em ambos os eixos). O eixo Y é invertido porque o SDL usa o canto superior esquerdo como origem, enquanto o OpenGL usa o canto inferior esquerdo.

5.16 CMake e Build System

5.16.1 Estrutura

O CMakeLists.txt está organizado em:

- Configuração inicial (versão, standard C++, flags);

- Adição de subdiretórios (vendedor/SDL3, vendedor/SDL_ttf);
- Criação da biblioteca GLAD (ou inclusão direta);
- Definição de ENGINE_SOURCES;
- Criação do executável Game;
- Configuração de *include paths*;
- Linkagem de bibliotecas;
- Cópia de DLLs no Windows.

5.16.2 Configuração Inicial

```
1 cmake_minimum_required(VERSION 3.25)

2 project(EngineProject VERSION 0.1.0 LANGUAGES C CXX)

3

4 set(CMAKE_CXX_STANDARD 23)

5 set(CMAKE_CXX_STANDARD_REQUIRED ON)

6 set(CMAKE_CXX_EXTENSIONS OFF)

7 set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

8

9 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")

10   add_compile_options(-Wall -Wextra -Wpedantic)

11 endif()
```

Optou-se por `CMAKE_CXX_EXTENSIONS OFF` para garantir portabilidade (não usa extensões específicas do GCC). As flags `-Wall -Wextra -Wpedantic` garantem que todos os avisos são visíveis, o que ajudou a detetar problemas ao longo do desenvolvimento.

5.16.3 Cópia de DLLs

O CMake não copia automaticamente as DLLs (`SDL3.dll`, `SDL3_ttf.dll`) para a pasta do executável. A solução foi usar `add_custom_command` com *generator expressions*:

5.16.4 Comandos de Build

Windows:

```
cmake -S . -B build\windows -G Ninja
ninja -C build\windows
```

Linux:

```
cmake -S . -B build\windows -G Ninja
ninja -C build\linux
```

Ambos os comandos são executados dentro da pasta raiz do projeto.

5.17 Tratamentos de erro e Robustez

5.17.1 Estratégia Adotada

O motor adota uma estratégia de **falha controlada**: sempre que um subsistema falha, o erro é registado e o motor continua em execução (quando possível) ou termina de forma limpa (quando inevitável).

5.17.2 Tipos de erro

A Tabela 7 classifica os erros por gravidade e comportamento:

Tabela 7- Classificação de erros

Tipo	Exemplo	Comportamento
Fatal	Falha na criação do contexto OpenGL	return false, motor termina
Recuperável	Textura em falta	Log::Warning, entidade não renderizada
Aviso	Gamepad não detetado	Log::Warning, motor continua sem gamepad

A distinção entre erros fatais e recuperáveis é importante: um erro fatal impede o funcionamento do motor (ex: sem contexto OpenGL, não há renderização); um erro recuperável permite que o motor continue, apenas com funcionalidade reduzida (ex: sem textura, o sprite não é desenhado, mas o resto funciona).

5.17.3 Logging como Ferramenta de Diagnóstico

O sistema de logging é a principal ferramenta de diagnóstico do motor. Os níveis (Info, Warning, Error) permitem filtrar mensagens por gravidade, e o *throttling* evita spam.

Em caso de erro, a mensagem inclui:

- A descrição do erro;
- A localização (ficheiro e linha, quando aplicável);
- A mensagem de erro do SDL/OpenGL (quando disponível).

Exemplo de Log (Início de encerramento da aplicação):

```
[2026-08-27 17:24:21.315] [INFO] FPS: 60.2 | Delta: 0.0168s | Elapsed: 6.05s  
[2026-08-27 17:24:21.647] [WARNING] Quit event received. Closing window.  
[2026-08-27 17:24:21.663] [INFO] Shutting down Application...
```

6. Resultados e Discussão

6.1 Introdução

Este capítulo apresenta os resultados obtidos ao longo do desenvolvimento do motor de jogo, analisa-os criticamente à luz dos objetivos definidos no Capítulo 1, e discute as limitações e ameaças à validade do trabalho. Segue a lógica recomendada pelas guidelines institucionais: apresentar a evidência antes da interpretação, ligar os resultados aos objetivos, e assumir limitações com maturidade crítica.

O capítulo está organizado em sete secções: (6.2) resultados obtidos, com métricas de desempenho e cumprimento de requisitos; (6.3) análise crítica, distinguindo o que correu melhor e pior do que o esperado; (6.4) comparação com o estado da arte; (6.5) limitações técnicas, de escopo e metodológicas; (6.6) ameaças à validade interna e externa; e (6.7) lições aprendidas.

6.2 Resultados Obtidos

6.2.1 Estado Final do Motor

No final do período de desenvolvimento, o motor de jogo encontra-se num estado funcional e estável. A Tabela 9 resume o estado dos principais subsistemas.

Tabela 8-Estado final dos subsistemas do motor

Subsistema	Estado	Observações
Janela e contexto OpenGL	Completo	Suporte a redimensionamento e fullscreen
Renderização 2D	Completo	Sprites texturizados, tiling, transparência
ECS (EnTT)	Completo	Componentes e sistemas desacoplados
Input	Completo	Teclado, rato e gamepad
Áudio	Completo	Efeitos WAV com conversão automática
Eventos	Completo	EventBus centralizado
Cenas	Completo	SceneManager com pilha de cenas
Menu	Parcial	Opções "Play" e "Exit" funcionais; "Options" não implementada
Demo2DFixedScene	Completo	Plataforma com inimigos, colisões e stomping
Documentação Doxygen	Completo	Documentação HTML navegável

6.2.2 Desempenho

O desempenho do motor foi medido através do `FpsCounter`, que regista periodicamente (a cada segundo) o número de frames por segundo, o delta time e o tempo decorrido.

Os resultados vão ao encontro dos valores previstos nos requisitos não-funcionais (RNF01: ≥ 60 FPS com vsync). O tempo de arranque é praticamente impercetível ao utilizador, e o uso de memória é adequado para uma aplicação de desktop moderna.

O motor utiliza `SDL_GL_SetSwapInterval(1)` para ativar a sincronização vertical, o que limita o FPS à taxa de atualização do monitor. Esta decisão evita *screen tearing* e reduz o consumo energético, ao custo de introduzir uma latência ligeiramente superior entre input e renderização. Em monitores de 144 Hz, o motor atinge os 144 FPS, demonstrando que o limite é imposto pelo hardware e não pelo motor.

O delta time é calculado com `SDL_GetPerformanceCounter`, que oferece precisão de microssegundos. Esta precisão é suficiente para jogos 2D, mas para simulações físicas mais complexas poderia ser necessário um *fixed timestep* (não implementado).

Os ~80 MB de RAM incluem o carregamento completo dos assets (texturas, sons e fontes) em memória. Para um motor 2D com o escopo atual, este valor é adequado. Em cenas com muitas texturas de alta resolução, seria necessário implementar carregamento dinâmico (*streaming*) e descarregamento de recursos não utilizados.

6.2.3 Cumprimento dos Requisitos Funcionais

Dos 15 requisitos funcionais, 14 foram completamente cumpridos e 1 ficou parcialmente implementado (RF12 – Menu). Esta taxa de cumprimento é consistente com o planeamento iterativo adotado, no qual o escopo foi ajustado à medida que o tempo disponível se tornava mais claro.

A opção "Options" do menu não foi implementada por uma decisão consciente de priorização: preferiu-se garantir que o núcleo do motor, a documentação Doxygen e o relatório final ficassem completos, em vez de adicionar uma funcionalidade não essencial. Esta decisão está documentada na secção 6.3.4 (o que correu pior do que o esperado).

6.2.4 Cumprimento dos Requisitos Não-Funcionais

Todos os requisitos não-funcionais foram cumpridos. O RNF05 (manutenibilidade) foi validado indiretamente: ao longo do desenvolvimento, várias funcionalidades foram adicionadas sem exigir alterações profundas na arquitetura, o que sugere baixo acoplamento. O RNF06 (extensibilidade) foi validado através da adição do sistema de inimigos sem alterar o `RenderSystem` ou o `ECSScene`.

6.3 Análise Crítica

6.3.1 Pontos Fortes

- **Arquitetura modular bem definida.** A separação em módulos independentes (`core`, `graphics`, `renderer`, `scene`, `systems`, `components`, `audio`, `events`, `resources`, `diagnostics`, `math`) permitiu que cada parte do motor evoluísse de forma relativamente isolada. Quando foi necessário alterar o sistema de áudio, por exemplo, apenas o módulo `audio/` foi afetado; o resto do motor continuou a funcionar sem alterações. Esta modularidade foi essencial para gerir a complexidade do projeto.
- **ECS aplicado corretamente com EnTT.** O padrão Entidades-Componentes-Sistemas foi implementado de forma consistente, com todos os componentes em namespace `Components` e os sistemas a processarem *views* filtradas. A adição de novos tipos de entidades (ex: inimigos) não exigiu alterações ao código existente – apenas novos componentes e sistemas. O `RenderSystem`, por exemplo, processa qualquer entidade que tenha `Transform` e `SpriteRenderer`, independentemente do tipo concreto.
- **Documentação Doxygen completa.** Todo o código está documentado com Doxygen, incluindo `@brief`, `@param`, `@return` e `@see`. A documentação HTML gerada é navegável e permite compreender a arquitetura sem ler o código-fonte. Esta documentação foi produzida incrementalmente, à medida que cada classe era implementada, o que reduziu o esforço final.
- **Cena de demonstração funcional.** A `Demo2DFixedScene` valida a integração de todos os subsistemas: renderização, ECS, física, input, áudio e cenas. Sem ela, seria

impossível demonstrar que o motor funciona como um todo. A cena inclui jogador controlável, plataformas, inimigos com patrulha, colisões AABB e mecânica de stomping.

- **Logging detalhado.** O sistema de logging permitiu identificar rapidamente erros de carregamento de recursos, falhas de shaders e problemas de inicialização de subsistemas. A funcionalidade de *throttling* (InfoThrottled) evitou que mensagens repetidas inundassem o log, mantendo-o legível. Sem este sistema, a depuração teria sido muito mais lenta.
- **Portabilidade.** O motor compila e corre em Windows e Linux sem alterações ao código-fonte. Esta portabilidade foi validada ao longo do desenvolvimento, não apenas no final. A abstração fornecida pelo SDL3 e o uso de CMake foram fundamentais para este resultado.

6.3.2 Pontos Fracos

- **Menu incompleto.** A opção "Options" não foi implementada. Embora o menu funcione (navegação por teclado, rato e gamepad), falta a opção intermédia. Esta limitação foi uma decisão consciente de priorização: preferiu-se garantir que o núcleo do motor e a documentação ficassem completos.
- **Sem animações de sprites.** O motor apenas suporta sprites estáticos. Não há suporte a *sprite sheets* nem a interpolação de frames. Esta limitação reduz o apelo visual dos jogos criados com o motor e obriga a que cada estado de uma personagem seja uma textura separada.
- **Sem sistema de partículas.** Não há suporte a efeitos visuais como fumo, faíscas ou explosões. Esta funcionalidade é comum em motores modernos e a sua ausência limita a expressividade visual dos jogos.
- **Física simplificada.** O sistema de física limita-se a colisões AABB (*Axis-Aligned Bounding Box*), sem resposta a colisões complexas, sem rotação, sem forças. Para

jogos mais complexos (ex: plataformas com rampas, colisões circulares), seria insuficiente. A integração de uma biblioteca como Box2D resolveria esta limitação.

- **Sem tilemaps.** Os níveis são criados programaticamente, entidade a entidade. Não há suporte para carregar mapas de ficheiros (ex: formato Tiled). Isto torna a criação de níveis trabalhosa e propensa a erros.
- **Sem editor de níveis.** Não há interface gráfica para criar níveis. Toda a configuração é feita em código, o que limita a iteração rápida sobre o design dos níveis.
- **Sem música de fundo.** Apenas efeitos sonoros curtos são suportados. Não há *streaming* de música longa (OGG/MP3). Para jogos mais completos, seria necessário implementar *streaming* e mistura de canais.
- **Sem multithreading.** Toda a lógica corre na thread principal. Não há paralelização de sistemas, o que limita o desempenho em cenários com muitas entidades. Em cenas com centenas de inimigos, o FPS degradar-se-ia significativamente.
- **Sem testes unitários.** A validação foi feita manualmente, através da cena de demonstração. Regressões podem não ser detetadas imediatamente, especialmente em partes do código menos exercitadas.

6.3.3 Fatores de Sucesso

Apesar da incerteza inicial (nunca tinha integrado estas bibliotecas), a integração correu melhor do que o esperado. O SDL3 fornece uma API limpa para criação de contexto OpenGL (SDL_GL_CreateContext), e o GLAD simplificou o carregamento de funções (gladLoadGLLoader). A documentação oficial do SDL3 e o *Learn OpenGL* foram recursos essenciais.

O motor atinge os 1400-1500 FPS sem vsync, o que é bastante superior ao mínimo necessário (60 FPS). Este desempenho deve-se, em parte, à simplicidade da cena de demonstração, mas

também à eficiência do EnTT e do pipeline OpenGL. Mesmo com vsync ativo, o motor mantém 60 FPS estáveis em monitores de 60 Hz e 144 FPS em monitores de 144 Hz.

A compilação em Windows e Linux foi conseguida sem grandes dificuldades, graças à abstração fornecida pelo SDL3 e ao uso de CMake. A única diferença significativa foi a cópia de DLLs (específica do Windows), resolvida com `add_custom_command`.

O motor não sofreu *crashes* em uso normal. Os problemas encontrados (GLAD, DLLs, namespaces, texto invertido, áudio) foram todos resolvidos antes da fase final, e a cena de demonstração corre de forma estável durante horas. Este resultado deve-se, em parte, ao uso de *smart pointers* e à verificação sistemática de erros (via `Log::Error` e retornos antecipados).

6.3.4 Bloqueios e Dificuldades

O texto aparecia invertido verticalmente e desfocado. A resolução exigiu compreender o *flip* de superfícies SDL antes de as carregar para OpenGL, e ajustar a filtragem de texturas (GL_NEAREST para magnificação, GL_LINEAR para minificação). Foi um problema frustrante, mas educativo.

O SDL3 abre o dispositivo com uma especificação diferente da pedida (ex: 48 kHz em vez de 44.1 kHz), o que causou problemas iniciais de reprodução. A solução (SDL_GetAudioDeviceFormat) é simples, mas exigiu compreender o funcionamento interno do SDL3 e da API SDL_AudioStream.

O CMake não copia automaticamente as DLLs (SDL3.dll, SDL3_ttf.dll) para a pasta do executável. Foram necessárias três tentativas até encontrar a solução correta com *generator expressions* (`$<TARGET_FILE:SDL3-shared>`).

Inicialmente, o plano previa um menu completo com transições e opções configuráveis. O tempo disponível não permitiu, e o escopo foi reduzido para "Play" e "Exit". Esta redução foi documentada como limitação em 6.5.3.

O tempo de desenvolvimento efetivo (2 meses) foi inferior ao planeado (2–3 meses de planeamento + 2 meses de desenvolvimento), o que obrigou a reduções de escopo. A falta de experiência prévia com as bibliotecas contribuiu para esta compressão.

6.4 Comparação com o Estado da Arte

6.4.1 Comparação com Motores Comerciais

Comparar o presente motor com Unity, Unreal Engine ou Godot é, à partida, uma comparação desigual. Estes motores têm anos (ou décadas) de desenvolvimento, equipas de centenas de pessoas e comunidades de milhões de utilizadores. O presente motor é um exercício pedagógico, não uma alternativa. No entanto, a comparação é útil para identificar o que falta e o que foi bem feito. A Tabela 9 compara as funcionalidades.

Tabela 9- Comparação com motores comerciais

Funcionalidade	Motor atual	Godot 4.x	Unity 2D	Unreal
Renderização 2D	☑	☑	☑	☑
ECS	☑	☑	⚠ DOTS	☒
Animações de sprites	☒	☑	☑	☑
Sistema de partículas	☒	☑	☑	☑
Tilemaps	☒	☑	☑	⚠
Editor visual	☒	☑	☑	☑
Física robusta	☒	☑	☑	☑

Funcionalidade	Motor atual	Godot 4.x	Unity 2D	Unreal
Multithreading	✗	✓	✓	✓
Documentação	✓	✓	✓	✓
Open-source	✓	✓	✗	✗

A comparação evidencia que o motor está longe de competir em funcionalidades. No entanto, cumpre o objetivo acadêmico de demonstrar princípios arquiteturais sólidos. Em relação ao **Godot 4.x**, que também usa ECS, a diferença principal é a maturidade: o Godot tem anos de desenvolvimento e uma comunidade enorme. O presente motor é um exercício pedagógico, não uma alternativa.

6.4.2 Comparação com Projetos Acadêmicos Similares

Não foi possível identificar, na literatura consultada, projetos acadêmicos diretamente comparáveis (mesmo escopo, mesmas bibliotecas, mesmo prazo). Esta ausência é, em si, uma lacuna identificada: a maioria dos projetos acadêmicos ou usa motores existentes (Unity, Godot), ou foca-se em aspetos específicos (ex: um sistema de física), não no motor como um todo.

Esta lacuna reforça a contribuição do presente trabalho: documentar o processo completo de construção de um motor de jogo, incluindo as dificuldades e as soluções. A secção 1.3 (Problemas) já identificava esta lacuna; o presente capítulo confirma-a com evidência concreta.

6.5 Limitações

6.5.1 Limitações Técnicas

- **Sem multithreading.** Toda a lógica corre na thread principal. Em cenários com muitas entidades (ex: centenas de inimigos), o desempenho degradar-se-ia. A paralelização de sistemas (ex: física numa thread, renderização noutra) exigiria sincronização cuidadosa e não foi implementada. Esta é, provavelmente, a limitação técnica com maior impacto potencial no desempenho.
- **Sem batching.** Cada sprite é um *draw call* separado. Em cenas com muitas entidades, o número de *draw calls* cresce linearmente com o número de sprites, o que se torna um gargalo. O *batching* (agrupar sprites com a mesma textura num único *draw call*) reduziria este problema. Em cenas com centenas de entidades, a diferença de desempenho seria significativa.
- **Sem culling.** Todas as entidades são renderizadas, mesmo as que estão fora do ecrã. O *frustum culling* (descartar entidades fora da vista) melhoraria o desempenho em níveis grandes. Atualmente, uma entidade fora do ecrã consome o mesmo tempo de renderização que uma dentro.
- **Sem asset pipeline.** Os recursos são carregados em *runtime*, a partir de ficheiros PNG, WAV e TTF. Não há um pipeline de conversão para formatos otimizados (ex: *texture atlas*, compressão de áudio). Para projetos maiores, seria necessário um sistema de empacotamento de assets.
- **Sem sistema de física completo.** A física limita-se a AABB, sem forças, sem rotação, sem colisões contínuas. Para jogos mais complexos, seria necessário integrar uma biblioteca como Box2D. A arquitetura atual permitiria esta integração sem alterar o resto do motor, mas exigiria trabalho significativo.
- **Sem suporte a tilemaps.** Os níveis são criados programaticamente. Não há suporte para carregar mapas de ficheiros (ex: formato Tiled). Isto torna a criação de níveis trabalhosa e limita a escalabilidade do motor para jogos com níveis grandes.

- **Sem testes unitários.** Não há testes automáticos. A validação foi feita manualmente, através da cena de demonstração. Isto significa que regressões podem não ser detetadas imediatamente, especialmente em partes do código menos exercitadas.

6.5.2 Limitações de Escopo

- **Apenas 2D.** O motor não suporta 3D. Esta decisão foi consciente: um motor 3D exigiria um pipeline de renderização muito mais complexo (iluminação, câmaras perspectivas, gestão de profundidade), incompatível com o prazo disponível. No entanto, a estrutura de Transform já inclui coordenadas Z, e a Camera2D poderia ser complementada por uma Camera3D sem alterar o resto do motor.
- https://loshkinoleg.github.io/cheatsheets/CMakeLists_Cheatsheet
- <https://daringfireball.net/projects/markdown/basics>
- <https://www.richardlord.net/blog/ecs/what-is-an-entity-framework>
- <https://www.geeksforgeeks.org/java/single-responsibility-principle-in-java-with-examples/>
- <https://softwareengineering.stackexchange.com/questions/15515/when-to-use-assertions-and-when-to-use-exceptions>
- <https://learn.microsoft.com/en-us/visualstudio/debugger/c-cpp-assertions?view=visualstudio>
- <https://dev.to/dsaghliani/understanding-delta-time-in-games-3olf>
- <https://www.geeksforgeeks.org/cpp/constructors-c/>
- <https://www.cppbetterexplained.com/posts/cpp-lambda-functions/><https://stackoverflow.com/questions/14399929/should-i-use-public-or-private-variables>
- <https://semver.org/>

- <https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging>
- <https://git-scm.com/book/en/v2/Git-Tools-Submodules>
- https://en.wikipedia.org/wiki/UV_mapping
- <https://gamedev.stackexchange.com/questions/72044/why-do-we-use-4x4-matrices-to-transform-things-in-3d>
- <https://gamedev.stackexchange.com/questions/12146/polling-vs-event-driven-input>
- https://www.reddit.com/r/gamedev/comments/4ty1sg/handling_input_polling_or_event_driven_which_to/
- <https://machinations.io/glossary/dead-zone>
- https://en.wikipedia.org/wiki/Euler_angles
- <https://www.youtube.com/watch?v=zc8b2Jo7mno>
- <https://www.dataorienteddesign.com/dodbook/dodmain.html>
- <http://gameprogrammingpatterns.com/command.html>
- <https://github.com/Lawliet-create/Modern-CPP-Programming#1>
- https://github.com/2DGD-F0TH/2DGD_F0TH
- **Apenas WAV.** O motor não suporta OGG nem MP3. O SoundManager está preparado para receber novos formatos, bastando adicionar um novo carregador. A limitação atual deve-se à simplicidade do formato WAV e à ausência de uma biblioteca de decodificação de áudio comprimido.
- **Apenas uma janela.** O motor suporta apenas uma janela por instância. Múltiplas janelas exigiriam uma gestão mais complexa de contextos OpenGL e de eventos SDL.

- **Apenas uma thread.** O motor não suporta renderização assíncrona nem processamento paralelo. Esta limitação está relacionada com o ponto 1 das limitações técnicas.

6.6 Ameaças à validade

6.6.1 Ameaças Internas

- **Hardware específico.** Os testes foram realizados em dois computadores: um com Intel Iris Xe (gráficos integrados) e outro com GPU dedicada. O desempenho noutro hardware (ex: CPUs mais antigas, GPUs de gama baixa) pode variar. No entanto, como o motor não usa funcionalidades gráficas avançadas, é expectável que corra em qualquer GPU com suporte a OpenGL 4.6.
- **Cena de demonstração simples.** A Demo2DFixedScene tem um valor muito baixo de entidades. O desempenho observado (1400-1500 FPS) não é representativo de cenários com centenas ou milhares de entidades. Em cenas maiores, o desempenho degradar-se-ia devido à ausência de *batching* e *culling*.
- **Medições informais.** As métricas de desempenho foram obtidas através do FpsCounter, que regista valores a cada segundo. Não foram usadas ferramentas de *profiling* profissionais (ex: RenderDoc, Tracy), pelo que não há dados detalhados sobre *bottlenecks*. As medições são indicativas, não rigorosas.

6.6.2 Ameaças Externas

- **Generalização limitada.** As conclusões aplicam-se a este contexto específico (projeto individual, prazo de 2 meses, bibliotecas específicas). Não é possível generalizar para outros projetos com contextos diferentes. Em particular, as dificuldades encontradas (GLAD, DLLs, namespaces) podem não ser representativas de outros programadores com mais experiência.

- **Reprodutibilidade dependente de versões.** O motor foi testado com versões específicas das bibliotecas (SDL3 3.4.0, EnTT 4.0.0, etc.). Versões mais recentes podem introduzir incompatibilidades. A ausência de testes automáticos agrava este risco.
- **Documentação incompleta.** Embora o código esteja documentado com Doxygen, faltam instruções detalhadas de *setup* para novos programadores. Um README mais completo reduziria esta barreira. Atualmente, um novo programador precisaria de compreender o CMake e as dependências antes de conseguir compilar o projeto.
- **Ausência de testes automáticos.** A reprodutibilidade dos testes manuais depende do autor. Sem testes automáticos, é difícil garantir que o comportamento se mantém consistente entre execuções. Esta limitação é particularmente relevante para a validação de regressões.

6.7 Lições Aprendidas

6.7.1 Lições Técnicas

- **CMake é poderoso mas complexo.** Pequenos erros (ex: a forma como as definições de compilação são propagadas entre targets) causam problemas difíceis de diagnosticar. A documentação do CMake é extensa, mas nem sempre clara sobre estes detalhes.
- **Namespaces consistentes são essenciais.** Em projetos grandes, a inconsistência de namespaces causa erros de compilação difíceis de localizar. A uniformização de todos os componentes em Components foi uma decisão acertada, tomada após os primeiros erros de compilação.
- **Logging é indispensável.** Sem um sistema de logging, a depuração de problemas de carregamento de recursos ou de shaders seria muito mais lenta. A funcionalidade de *throttling* revelou-se particularmente útil para evitar spam de mensagens repetidas.

- **Doxygen compensa o esforço.** A documentação gerada ajuda a navegar no código e a compreender a arquitetura, mesmo para o próprio autor (que esquece detalhes ao longo do tempo). O esforço de documentar incrementalmente foi recompensado na fase de escrita do relatório.
- **O Git é mais do que controlo de versões.** O Git foi usado como ferramenta ativa de desenvolvimento, permitindo reverter tentativas falhadas e manter um registo cronológico do progresso. O histórico de commits é, em si, uma forma de documentação do processo.

6.7.2 Lições Metodológicas

- **Desenvolvimento iterativo funciona.** A organização em incrementos semanais permitiu validar cada funcionalidade antes de avançar, reduzindo o risco de acumular erros. Cada semana terminava com uma demo funcional, o que mantinha a motivação e permitia detetar problemas cedo.
- **Desenvolvimento exploratório com Git é viável.** Implementar diretamente no repositório principal, com o Git como rede de segurança, revelou-se mais eficiente do que criar protótipos isolados. O código é testado no ambiente final, sem duplicação, e cada commit documenta uma tentativa bem-sucedida.
- **Adaptação contínua é necessária.** O escopo do menu foi reduzido quando se tornou claro que o tempo não era suficiente. Esta decisão, embora difícil, permitiu garantir que o núcleo do motor ficasse completo. A capacidade de adaptar o plano às circunstâncias é uma competência essencial em engenharia de software.
- **A ausência de metodologia formal não é um problema.** Desde que haja disciplina (incrementos, validação, registo), é possível desenvolver de forma estruturada sem adotar Scrum ou Kanban. O importante é ter um processo claro, não um rótulo.

7. Conclusão e Trabalho Futuro

7.1 Síntese do Trabalho

O presente projeto teve como objetivo geral o desenvolvimento de um motor de jogo 2D modular, implementado em C++23, com recurso às bibliotecas SDL3 (para a camada de plataforma, input, áudio e eventos) e OpenGL 4.6 Core Profile (para a renderização acelerada por hardware), complementadas pelo carregador GLAD, pela biblioteca de ECS EnTT e pela biblioteca stb_image para carregamento de texturas.

O projeto foi realizado no âmbito da unidade curricular de Projeto de Software, do terceiro ano da Licenciatura em Gestão de Sistemas e Computação, ao longo de um semestre letivo.

A motivação central do trabalho prendeu-se com a necessidade de compreender, na prática, os princípios arquiteturais que sustentam motores comerciais como Unity, Unreal Engine ou Godot. Enquanto a maioria dos recursos disponíveis foca-se na utilização de motores existentes, este projeto propôs-se a construir um motor de raiz, confrontando o autor com decisões arquiteturais fundamentais: como estruturar o loop principal, como organizar as entidades de jogo, como desenhar o pipeline de renderização e como desacoplar os subsistemas.

Os objetivos específicos definidos no início do projeto foram integralmente cumpridos. A arquitetura modular foi implementada, organizada em onze módulos independentes (core, graphics, renderer, scene, systems, components, audio, events, resources, diagnostics e math), demonstrando os princípios de separação de responsabilidades e baixo acoplamento. O padrão Entidades-Componentes-Sistemas (ECS) foi adotado com a biblioteca EnTT, permitindo que novas entidades fossem adicionadas sem alterar o código existente. O sistema de renderização 2D foi implementado com suporte a texturas, tiling, transparência e renderização de texto com SDL_ttf. O sistema de áudio foi implementado com suporte a efeitos sonoros WAV e conversão automática de formatos. O sistema de eventos foi implementado com um EventBus centralizado, que desacopla emissores e recetores. O sistema de gestão de cenas foi implementado com um SceneManager que mantém uma pilha de cenas e permite transições controladas.

A cena de demonstração (Demo2DFixedScene) validou a integração de todos os subsistemas, implementando um pequeno plataforma com jogador controlável, plataformas estáticas e flutuantes, inimigos com patrulha horizontal, colisões AABB e mecânica de stomping ao estilo Super Mario. Esta cena demonstrou que o motor funciona como um todo integrado, não apenas como um conjunto de módulos isolados.

Os resultados obtidos demonstram a viabilidade da abordagem, com um desempenho estável a 60 FPS (com vsync ativo) e 1400-1500 FPS (sem vsync), e um tempo de arranque inferior a um segundo. Estes valores vão ao encontro dos requisitos não-funcionais definidos no Capítulo 4.

Das 15 requisitos funcionais definidos, 14 foram completamente cumpridos e 1 ficou parcialmente implementado (o menu, que ficou sem a opção "Options"). Esta taxa de cumprimento é consistente com o planeamento iterativo adotado, no qual o escopo foi ajustado à medida que o tempo disponível se tornava mais claro.

7.2 Contribuições

As principais contribuições deste trabalho podem ser organizadas em quatro categorias: contribuições técnicas, contribuições documentais, contribuições metodológicas e contribuições para trabalho futuro.

7.2.1 Contribuições Técnicas

- **Uma base de código reutilizável para jogos 2D.** O motor foi desenhado desde o início com a extensibilidade em mente. A separação em módulos independentes, o uso de abstrações (ex: GraphicsContext, Renderer, Scene) e a adoção do padrão ECS significam que o motor pode ser reutilizado para diferentes tipos de jogos 2D sem alterações profundas na arquitetura. A integração de uma biblioteca como Box2D para física, por exemplo, implicaria alterações apenas no módulo systems/.
- **Um exemplo prático de aplicação de ECS em C++ moderno.** Embora o ECS seja um padrão amplamente discutido na literatura, exemplos práticos de implementação

em C++ moderno (com smart pointers, namespace organizado, e integração com bibliotecas como EnTT) são menos comuns. O presente projeto fornece um exemplo concreto e funcional.

- **Uma camada de abstração gráfica portátil.** A separação entre a lógica de jogo e a API gráfica (OpenGL) significa que substituir OpenGL por Vulkan ou Direct3D 12 implicaria reescrever apenas a pasta `graphics/`, mantendo intactos o ECS, os sistemas, as cenas e a lógica de jogo. Esta abstração é uma contribuição técnica relevante para futuros desenvolvimentos.
- **Um sistema de eventos desacoplado e reutilizável.** O EventBus implementado é genérico e pode ser reutilizado em qualquer projeto que necessite de comunicação desacoplada entre subsistemas. A subscrição por tipo (`std::type_index`) é uma abordagem eficiente e type-safe.

7.2.2 Contribuições Documentais

- **Documentação Doxygen completa e navegável.** Todo o código está documentado com Doxygen, incluindo `@brief`, `@param`, `@return` e `@see`. A documentação HTML gerada é navegável e cobre todas as classes, métodos e ficheiros. Esta documentação é um recurso valioso para quem queira compreender ou estender o motor.
- **Registo transparente das dificuldades e soluções.** Ao longo do relatório, as dificuldades encontradas (GLAD, DLLs, namespaces, texto invertido, áudio) são documentadas em detalhe, com as soluções adotadas. Este registo é uma contribuição importante para futuros programadores que enfrentem problemas semelhantes.
- **Síntese metodológica.** A descrição da abordagem metodológica adotada (desenvolvimento iterativo e incremental, desenvolvimento exploratório com Git, adaptação contínua) pode servir de guia para futuros projetos individuais com características semelhantes (incerteza técnica elevada, prazo fixo, ausência de equipa).

7.2.3 Contribuições Metodológicas

- **Validação da abordagem híbrida pragmática.** O projeto demonstra que é possível desenvolver software de forma estruturada sem adotar uma metodologia formal única. Os três pilares adotados (desenvolvimento iterativo, desenvolvimento exploratório com Git, adaptação contínua) revelaram-se adequados ao contexto.
- **Evidência da viabilidade do desenvolvimento exploratório com Git.** A abordagem de implementar diretamente no repositório principal, usando o Git como rede de segurança, revelou-se mais eficiente do que a prototipagem isolada, para este contexto específico.

7.2.4 Contribuições para Trabalho Futuro

- **Uma base sólida para desenvolvimentos futuros.** O motor atual, embora limitado em funcionalidades, fornece uma base sólida sobre a qual se podem construir funcionalidades mais avançadas (animações, física robusta, tilemaps, editor de níveis). As limitações identificadas no Capítulo 6 são, em si, um roteiro para trabalho futuro.
- **Identificação de lacunas na literatura.** A ausência de projetos acadêmicos diretamente comparáveis (mesmo escopo, mesmas bibliotecas, mesmo prazo) é, em si, uma lacuna identificada. O presente trabalho contribui para preencher essa lacuna.

7.3 Limitações Reconhecidas

Apesar do cumprimento dos objetivos, o trabalho tem limitações que importa reconhecer:

- **Menu incompleto:** a opção "Options" não foi implementada;
- **Sem animações de sprites:** apenas sprites estáticos são suportados;
- **Sem sistema de partículas:** não há suporte a efeitos visuais;
- **Física simplificada:** apenas colisões AABB, sem forças ou rotação;
- **Sem tilemaps:** os níveis são criados programaticamente;

- **Sem editor de níveis:** toda a configuração é feita em código;
- **Sem música de fundo:** apenas efeitos sonoros curtos;
- **Sem multithreading:** toda a lógica corre na thread principal;
- **Sem testes unitários:** a validação foi manual.

Estas limitações estão detalhadas no Capítulo 6 (secção 6.5) e são reconhecidas como consequência do escopo e do tempo disponível, não como falhas de arquitetura.

7.4 Trabalho Futuro

O motor desenvolvido é uma base sólida, mas está longe de ser um produto completo. Propõe-se, como desenvolvimentos futuros, um conjunto de melhorias organizadas por prioridade e complexidade.

7.4.1 Curto Prazo (Alta Prioridade)

- **Sistema de animação de sprites.** Suporte a *sprite sheets* e interpolação de frames, permitindo que as personagens tenham animações (andar, saltar, atacar). Esta é, provavelmente, a funcionalidade mais impactante para o apelo visual dos jogos. A implementação envolveria a adição de um componente Animator e de um sistema AnimationSystem.
- **Suporte a tilemaps (formato Tiled).** Carregamento de mapas criados no editor Tiled, permitindo a criação de níveis maiores e mais complexos sem código. A implementação envolveria a adição de um carregador de ficheiros TMX e de um sistema de renderização otimizado para tiles.
- **Testes unitários.** Cobertura com Catch2 ou GoogleTest, garantindo que o comportamento do motor se mantém consistente entre alterações. Os testes mais prioritários seriam para o ECS, o sistema de eventos e o sistema de áudio.

7.4.2 Médio Prazo (Prioridade Média)

- **Motor de física robusto.** Integração com Box2D, permitindo colisões complexas, forças, rotação e *joints*. A arquitetura atual já isola a física do resto do motor, pelo que a integração seria relativamente contida.
- **Editor de níveis.** Interface gráfica para criação de níveis, arrastando e soltando entidades, definindo propriedades e testando o nível em tempo real. Esta é uma funcionalidade complexa, mas transformadora para a usabilidade do motor.
- **Sistema de partículas.** Suporte a efeitos visuais como fumo, faíscas, explosões e chuva. A implementação envolveria um componente ParticleEmitter e um sistema dedicado.
- **Música de fundo com streaming.** Suporte a ficheiros OGG/MP3 com *streaming*, permitindo música longa sem carregar tudo em memória. A implementação envolveria a integração de uma biblioteca como stb_vorbis ou miniaudio.

7.4.3 Longo Prazo (Baixa Prioridade)

- **Batching de renderização.** Otimização do RenderSystem para agrupar sprites com a mesma textura num único *draw call*, reduzindo o número de chamadas à GPU. Esta otimização é relevante apenas em cenas com muitas entidades.
- **Multithreading.** Paralelização de sistemas (ex: física numa thread, renderização noutra), melhorando o desempenho em cenários com muitas entidades. A implementação exigiria sincronização cuidadosa e uma revisão da arquitetura atual.
- **Suporte a 3D.** Extensão do motor para 3D, adicionando uma Camera3D, um sistema de iluminação e gestão de profundidade. A estrutura de Transform já inclui coordenadas Z, pelo que a base está preparada.

7.5 Reflexão Final

O desenvolvimento deste motor de jogo foi um percurso exigente mas profundamente enriquecedor. Permitiu consolidar conhecimentos avançados de C++ (smart pointers, move semantics, templates, namespace), compreender a arquitetura interna de um motor de jogo, e aplicar princípios de engenharia de software num contexto real.

As dificuldades encontradas — longe de serem obstáculos — foram oportunidades de aprendizagem. Cada erro de compilação, cada *bug* de renderização, cada problema de integração, contribuíram para uma compreensão mais profunda dos sistemas envolvidos.

O motor resultante está longe de ser um produto comercial, mas é uma base sólida sobre a qual se pode construir. Mais importante do que o produto final é o processo: a metodologia iterativa, a documentação contínua e a reflexão crítica sobre as decisões tomadas. Estes são os ativos que permanecem para além do projeto.

A nível pessoal, o projeto reforçou a convicção de que a engenharia de software é, acima de tudo, uma disciplina de gestão de complexidade. A modularidade, a abstração e a documentação não são fins em si mesmos – são ferramentas para tornar sistemas complexos compreensíveis e manuteníveis. Esta lição aplica-se a qualquer projeto de software, não apenas a motores de jogo.

Em suma, o presente projeto cumpriu os objetivos a que se propôs, deixou um conjunto de contribuições verificáveis, e estabeleceu uma base sólida para trabalho futuro.

Referências

de Vries, J. (2024). *Learn OpenGL*. Learn OpenGL. <https://learnopengl.com>

Fabian, R. (2018). *Data-oriented design: Software engineering for limited resources and short schedules*. <https://www.dataorienteddesign.com/dodbook/>

Gregory, J. (2018). *Game engine architecture* (3rd ed.). CRC Press.

Doxygen. (2024). *Doxygen* (Version 1.18) [Computer software]. <https://www.doxygen.nl>

Newzoo. (2024, August). *Global games market report 2024*. https://newzoo.com/.https://best-of-gaming.be/wp-content/uploads/2024/09/2024_Newzoo_Global_Games_Market_Report.pdf

AmbroggioMusic. (2026). *Jump SFX by AmbroggioMusic [Sound Effects]*. AmbroggioMusic. <https://ambroggiomusic.itch.io/jump-sfx>

Skypjack. (2026). *EnTT* (Version 4.0.0) [Computer software]. GitHub. <https://github.com/skypjack/entt>

Herberth, D. (2024). *GLAD* (Version 0.1.36) [Computer software]. <https://glad.dav1d.de>

Kitware. (2024). *CMake* (Version 4.4.3) [Computer software]. <https://cmake.org>

Ninja-build. (2025). *Ninja* (Version 1.13.2) [Computer software]. <https://ninja-build.org>

Khronos Group. (2016). *OpenGL* (Version 4.6) [Computer software]. <https://www.khronos.org/opengl/>

libsdl-org. (2025). *SDL3 documentation*. <https://wiki.libsdl.org/SDL3>

Barrett, S. (2024). *stb_image* (Version 2.30) [Computer software]. GitHub. <https://github.com/nothings/stb>

Square Enix. (2025). *Final Fantasy Tactics – The Ivalice Chronicles* [Videogame]. Square Enix.

Apêndices

Apêndice A — Excertos de Código Relevantes

A.1 — Application::Initialize

```
1 bool Application::Initialize(){
2     //Initialize logging system
3     Log::Initialize("logs/");
4     Log::Info("Application: Initializing...");
5
6     //Initialize SDL with video and audio subsystems
7     if(SDL_Init(SDL_INIT_VIDEO | SDL_INIT_AUDIO) < 0){
8         Log::Error(std::format("SDL Init failed: {}", SDL_GetError()));
9         return false;
10    }
11
12    //Initialize SDL_ttf for font rendering
13    if(TTF_Init() != 0){
14        Log::Error(std::format("SDL_ttf initialization failed: {}", SDL_GetError()));
15    } else{
16        Log::Info("SDL_ttf initialized successfully.");
17    }
```

```
18
19 //Create the game window
20 if(!m_Engine.Initialize("Game Window", 1920, 1080, false)){
21     Log::Error("Failed to initialize Engine.");
22     return false;
23 }
24
25 //Subscribe to window close event
26 EventBus::GetInstance().Subscribe<WindowCloseEvent>([this](Event& e){
27     Log::Info("WindowCloseEvent received, shutting down game.");
28     m_Engine.SetRunning(false);
29 });
30
31 //Subscribe to window resize event
32 EventBus::GetInstance().Subscribe<WindowResizeEvent>([](Event& e){
33     WindowResizeEvent& resize = static_cast<WindowResizeEvent&>(e);
34     Log::Info(std::format("Window resized to {}x{}", resize.GetWidth(),
resize.GetHeight()));
35 });
36
37 //Set callback for unhandled SDL events
```

```
38  SDL_EventTranslator::SetUnhandledCallback([](const SDL_Event& event){
39      Log::Warning(std::format("Unhandled SDL event type: {}", event.type));
40  });
41
42  //Initialize OpenGL graphics context
43  m_Graphics = std::make_unique<GraphicsContext>();
44  if(!m_Graphics->Initialize()){
45      Log::Error("GraphicsContext initialization failed.");
46      return false;
47  }
48
49  //Initialize camera with aspect ratio
50  int winWidth, winHeight;
51  SDL_GetWindowSize(m_Engine.GetWindow().GetNativeWindow(), &winWidth,
&winHeight);
52  Input::SetWindowSize(winWidth, winHeight);
53  float aspect = static_cast<float>(winWidth) / static_cast<float>(winHeight);
54  float height = 5.0f;
55  float width = height * aspect;
56  m_Camera = std::make_unique<Camera2D>(-width, width, -height, height);
57  m_Camera->SetPosition(Vector3D(0.0f, 0.0f, 0.0f));
```

```
58
59 //Initialize game world boundaries
60 m_World = std::make_unique<GameWorld>(-20.0f, 20.0f, -10.0f, 10.0f);
61
62 //Initialize gamepad manager
63 if(!GamepadManager::GetInstance().Initialize()){
64     Log::Warning("Gamepad initialization failed. Continuing without gamepad
support.");
65 }
66
67 //Initialize FPS counter
68 m_FpsCounter = std::make_unique<FpsCounter>();
69
70 //Subscribe to window resize events for camera updates
71 EventBus::GetInstance().Subscribe<WindowResizeEvent>([this](Event& e){
72     WindowResizeEvent& resizeEvent = static_cast<WindowResizeEvent&>(e);
73     int newWidth = resizeEvent.GetWidth();
74     int newHeight = resizeEvent.GetHeight();
75
76     Input::SetWindowSize(newWidth, newHeight);
77
```

```
78     //Skip camera update for fixed camera scenes

79     Scene* currentScene = m_SceneManager.GetCurrentScene();

80     if(currentScene && currentScene->GetName() == "Demo2D_FixedCamera"){

81         Log::Info("Fixed camera scene active: skipping camera projection update.");

82         return;

83     }

84

85     if(m_Camera){

86         float aspect = static_cast<float>(newWidth) / static_cast<float>(newHeight);

87         float height = 5.0f;

88         float width = height * aspect;

89         m_Camera->SetProjection(-width, width, -height, height);

90     }

91 });

92

93 Log::Info("Application initialized successfully.");

94

95 //Initial scene - Demo2DFixedScene

96 auto initialScene = std::make_unique<Demo2DFixedScene>(

97     m_Graphics->GetShader(),
```

```

98             m_Graphics->GetVertexArray(),
99             m_Graphics->GetIndexBuffer(),
100             *m_Camera,
101             m_Graphics->GetLineShader()
102     );
103
104     return true;
105 }

```

A.2 — RenderSystem::Render

```

1 static void Render(entt::registry& registry, Shader& shader, const Matrix4& view,
2             const Matrix4& projection, VertexArray& va, IndexBuffer& ib){
3             auto view_entities = registry.view<Components::Transform,
Components::SpriteRenderer>();
4
5             for(auto [entity, transform, sprite] : view_entities.each()){
6                 //Skip entities without texture
7                 if(!sprite.Texture) continue;
8
9                 //Build model matrix: translation * rotation * scale
10                Matrix4 model = Matrix4::Translation(transform.Position);
11                model = model * Matrix4::RotationZ(transform.Rotation.z);

```

```
12     model = model * Matrix4::Scale(transform.Scale);
13
14     //Get tile scale (default to 1.0 if not present)
15     float tileScaleX = 1.0f;
16     float tileScaleY = 1.0f;
17     if(registry.all_of<Components::TileScale>(entity)){
18         const auto& ts = registry.get<Components::TileScale>(entity);
19         tileScaleX = ts.ScaleX;
20         tileScaleY = ts.ScaleY;
21     }
22
23     //Draw the textured quad
24     Renderer::DrawTexturedQuad(shader, va, ib, *sprite.Texture,
25                                 model, view, projection, tileScaleX, tileScaleY);
26 }
27 }
```

A.3 — EnemySystem::Update

```
1 static void Update(entt::registry& registry, float deltaTime,
2
3     const std::vector<entt::entity>& platforms, float worldMinY){
4
5     //View: entities with Transform, Enemy, Patrol, and PhysicsBody
6
7     auto view = registry.view<Components::Transform, Components::Enemy,
8
9     Components::Patrol, Components::PhysicsBody>();
10
11     for(auto [entity, transform, enemy, patrol, physics] : view.each()){
12
13         if(!enemy.IsActive) continue;
14
15
16         //Enemy patrol movement
17
18         float speed = patrol.Speed * deltaTime;
19
20         if(patrol.MovingRight){
21
22             transform.Position.x += speed;
23
24             if(transform.Position.x >= patrol.EndPosition.x) patrol.MovingRight = false;
25
26         } else{
27
28             transform.Position.x -= speed;
29
30             if(transform.Position.x <= patrol.StartPosition.x) patrol.MovingRight = true;
31
32         }
33     }
34 }
```

```
20    //Apply gravity

21    float gravity = -9.8f * physics.GravityScale;

22    physics.Velocity.y += gravity * deltaTime;

23    transform.Position.y += physics.Velocity.y * deltaTime;

24

25    //Collision detection and resolution with platforms

26    Vector3D enemyHalfSize = transform.Scale * 0.5f;

27    bool grounded = false;

28

29    for(entt::entity platform : platforms){

30        auto& platformTransform = registry.get<Components::Transform>(platform);

31        Vector3D platformHalfSize = platformTransform.Scale * 0.5f;

32

33        //Calculate AABB bounds

34        float enemyLeft = transform.Position.x - enemyHalfSize.x;

35        float enemyRight = transform.Position.x + enemyHalfSize.x;

36        float enemyBottom = transform.Position.y - enemyHalfSize.y;

37        float enemyTop = transform.Position.y + enemyHalfSize.y;

38

39        float platformLeft = platformTransform.Position.x - platformHalfSize.x;
```

```
40     float platformRight = platformTransform.Position.x + platformHalfSize.x;
41     float platformTop = platformTransform.Position.y + platformHalfSize.y;
42     float platformBottom = platformTransform.Position.y - platformHalfSize.y;
43
44     bool overlapX = (enemyRight > platformLeft && enemyLeft < platformRight);
45     bool overlapY = (enemyTop > platformBottom && enemyBottom <
platformTop);
46
47     if(overlapX && overlapY){
48         //Resolve collision
49         Vector3D delta = transform.Position - platformTransform.Position;
50         Vector3D overlap = enemyHalfSize + platformHalfSize -
51         Vector3D(std::abs(delta.x), std::abs(delta.y), 0.0f);
52
53         if(overlap.y < overlap.x){
54             //Vertical collision
55             if(delta.y > 0.0f){
56                 transform.Position.y += overlap.y;
57                 physics.Velocity.y = 0.0f;
58                 grounded = true;
59             } else{
```

```
60         transform.Position.y -= overlap.y;
61
62         if(physics.Velocity.y > 0.0f){
63             physics.Velocity.y = 0.0f;
64         }
65     } else{
66         //Horizontal collision
67
68         if(delta.x > 0.0f) transform.Position.x += overlap.x;
69         else transform.Position.x -= overlap.x;
70     }
71 }
72
73 //Clamp to world floor
74 float worldFloor = worldMinY + enemyHalfSize.y;
75 if(transform.Position.y < worldFloor){
76     transform.Position.y = worldFloor;
77     physics.Velocity.y = 0.0f;
78     grounded = true;
79 }
```

```

80
81     //Update grounded state
82     physics.IsGrounded = grounded;
83 }
84 }

```

A.4 — PlayerController::Update

```

1 void PlayerController::Update(){
2     if(m_IsDead) return;
3
4     float dt = Time::DeltaTime();
5     auto& transform = m_Registry.get<Components::Transform>(m_PlayerEntity);
6
7     //Get direction from keyboard
8     Vector2D keyboardDirection(0.0f, 0.0f);
9     if(Input::IsKeyHeld(SDL_SCANCODE_LEFT)) keyboardDirection.x -= 1.0f;
10    if(Input::IsKeyHeld(SDL_SCANCODE_RIGHT)) keyboardDirection.x += 1.0f;
11
12    //Normalize diagonal movement
13    if(keyboardDirection.x != 0.0f && keyboardDirection.y != 0.0f){
14        keyboardDirection = keyboardDirection * 0.7071f;

```

```
15  }  
  
16  
  
17  //Get gamepad left joystick direction  
  
18  Vector2D gamepadDirection(0.0f, 0.0f);  
  
19  bool gamepadActive = false;  
  
20  if(GamepadManager::GetInstance().IsConnected(0)){  
  
21      gamepadDirection.x = GamepadManager::GetInstance().GetAxis(0,  
SDL_GAMEPAD_AXIS_LEFTX);  
  
22  
  
23      float deadzone = 0.2f;  
  
24      if(std::abs(gamepadDirection.x) < deadzone) gamepadDirection.x = 0.0f;  
  
25      if(std::abs(gamepadDirection.y) < deadzone) gamepadDirection.y = 0.0f;  
  
26  
  
27      float magnitude = std::sqrt(gamepadDirection.x * gamepadDirection.x +  
gamepadDirection.y * gamepadDirection.y);  
  
28      if(magnitude > 0.1f){  
  
29          gamepadActive = true;  
  
30          gamepadDirection = gamepadDirection / magnitude;  
  
31      }  
  
32  }  
  
33
```

```
34 //Combine directions: gamepad takes priority over keyboard
35 Vector2D inputDirection(0.0f, 0.0f);
36 if(gamepadActive) inputDirection = gamepadDirection;
37 else inputDirection = keyboardDirection;
38
39 //Apply horizontal movement
40 float speed = m_Speed * dt;
41 if(inputDirection.x != 0.0f || inputDirection.y != 0.0f){
42     transform.Position.x += inputDirection.x * speed;
43     transform.Position.y += inputDirection.y * speed;
44 }
45
46 //Jump detection
47 bool wantToJump = false;
48 if(Input::IsKeyPressed(SDL_SCANCODE_SPACE)) wantToJump = true;
49 if(GamepadManager::GetInstance().IsButtonPressed(0,
SDL_GAMEPAD_BUTTON_EAST)) wantToJump = true;
50
51 //Execute jump
52 if(wantToJump && m_IsGrounded){
53     m_IsJumping = true;
```

```

54     m_IsGrounded = false;

55     m_JumpTimer = 0.0f;

56     m_Velocity.y = 12.0f;

57

58     //Dispatch jump event and play sound

59     PlayerJumpedEvent jumpEvent(transform.Position, m_Velocity.y);

60     EventBus::GetInstance().Dispatch(jumpEvent);

61     SoundManager::GetInstance().PlaySound("assets/sounds/jump.wav");

62     Log::Info("Jump started.");

63 }

64

65 //Vertical physics (jumping + gravity)

66 if(m_IsJumping){

67         if(Input::IsKeyHeld(SDL_SCANCODE_SPACE) ||
GamepadManager::GetInstance().IsButtonHeld(0, SDL_GAMEPAD_BUTTON_EAST)){

68         m_JumpTimer += dt;

69         float gravityReduction = 1.0f - (m_JumpTimer / 0.5f);

70         if(gravityReduction < 0.0f) gravityReduction = 0.0f;

71         m_Velocity.y += (gravityReduction * 0.5f - 9.8f) * dt;

72     } else{

73         m_Velocity.y = std::min(m_Velocity.y, 0.0f);

```

```
74     m_IsJumping = false;

75 }

76 } else{

77     if(!m_IsGrounded) m_Velocity.y -= 9.8f * dt;

78     else m_Velocity.y = 0.0f;

79 }

80

81 //Apply vertical velocity

82 if(!m_IsGrounded || m_Velocity.y > 0.0f) transform.Position.y += m_Velocity.y * dt;

83

84 //Landing detection and sound

85 if(m_IsGrounded && !m_WasGrounded){

86     SoundManager::GetInstance().PlaySound("assets/sounds/landing.wav");

87     Log::Info("Landed (sound played)");

88 }

89 m_WasGrounded = m_IsGrounded;

90

91 //Clamp player position to world boundaries

92 transform.Position = m_World.ClampPosition(transform.Position);

93
```

```

94  //Log player state for debugging

95  Log::InfoThrottled(std::format("Player grounded: {}, Is jumping: {}, Vertical velocity:
{:2f}",

96                                m_IsGrounded, m_IsJumping, m_Velocity.y),
"check_grounded_state", 2.0f);

97

98  //Death check - fell off the world

99  if(transform.Position.y < -15.0f) Die("Fell off the world");

100

101  if(m_IsGrounded) m_Velocity.y = 0.0f;

102 }

```

A.5 — Demo2DFixedScene::SetupScene

```

1 void Demo2DFixedScene::SetupScene(){

2  //Create ECS scene and world

3      m_ECSScene = std::make_unique<ECSScene>(m_Shader, m_VertexArray,
m_IndexBuffer);

4  m_World = std::make_unique<GameWorld>(-20.0f, 20.0f, -10.0f, 10.0f);

5

6  //Set camera projection to show the entire world

7  m_Camera.SetProjection(m_World->GetMinX(), m_World->GetMaxX(),

8                          m_World->GetMinY(), m_World->GetMaxY());

```

```
9
10 float centerX = (m_World->GetMinX() + m_World->GetMaxX()) * 0.5f;
11 float centerY = (m_World->GetMinY() + m_World->GetMaxY()) * 0.5f;
12 m_Camera.SetPosition(Vector3D(centerX, centerY, 0.0f));
13 m_Camera.Update();
14
15 Log::Info("Camera projection set to world bounds.");
16
17 //Load textures
18 auto playerTex = ResourceManager::GetInstance().LoadTexture("assets/textures/player.png");
19 auto tileTex = ResourceManager::GetInstance().LoadTexture("assets/textures/tile.png");
20 if(!playerTex || !tileTex){
21     Log::Error("Failed to load one or more textures.");
22     return;
23 }
24 auto wallTex = tileTex;
25
26 //Create player entity
27 m_PlayerEntity = m_ECSScene->CreateSpriteEntity(Vector3D(0.0f, 0.0f, 0.0f),
28                                                    Vector3D(3.0f, 3.0f, 1.0f),
```

```
29         playerTex);

30

31 //Create platform entities

32 //Left Wall

33 auto left_wall = m_ECSScene->CreateSpriteEntity(Vector3D(m_World->GetMinX() +
0.50f, 0.0f, 0.0f),
34
Vector3D(1.0f, m_World->GetMaxY() - m_World-
>GetMinY(), 1.0f),
35
wallTex);

36 m_PlatformEntities.push_back(left_wall);

37 m_ECSScene->GetRegistry().emplace<Components::TileScale>(left_wall, 1.0f, 20.0f);

38

39 //Right Wall

40 auto rgt_wall = m_ECSScene->CreateSpriteEntity(Vector3D(m_World->GetMaxX() -
0.50f, 0.0f, 0.0f),
41
Vector3D(1.0f, m_World->GetMaxY() - m_World-
>GetMinY(), 1.0f),
42
wallTex);

43 m_PlatformEntities.push_back(rgt_wall);

44 m_ECSScene->GetRegistry().emplace<Components::TileScale>(rgt_wall, 1.0f, 20.0f);

45

46 //Ceiling
```

```
47     auto ceiling = m_ECSScene->CreateSpriteEntity(Vector3D(0.0f, m_World-
>GetMaxY() - 0.50f, 0.0f),
48
         Vector3D(m_World->GetMaxX() - m_World->GetMinX(),
1.0f, 1.0f),
49
         wallTex);
50     m_PlatformEntities.push_back(ceiling);
51     m_ECSScene->GetRegistry().emplace<Components::TileScale>(ceiling, 39.0f, 1.0f);
52
53     //Floor
54     auto plt_floor = m_ECSScene->CreateSpriteEntity(Vector3D(0.0f, m_World-
>GetMinY() + 0.50f, 0.0f),
55
         Vector3D(m_World->GetMaxX() - m_World-
>GetMinX(), 1.0f, 1.0f),
56
         tileTex);
57     m_PlatformEntities.push_back(plt_floor);
58     m_ECSScene->GetRegistry().emplace<Components::TileScale>(plt_floor, 39.0f, 1.0f);
59
60     //Floating platform 1
61     auto plt_float1 = m_ECSScene->CreateSpriteEntity(Vector3D(-8.0f, -3.0f, 0.0f),
62
         Vector3D(4.0f, 1.0f, 1.0f),
63
         tileTex);
64     m_PlatformEntities.push_back(plt_float1);
```

```
65  m_ECSScene->GetRegistry().emplace<Components::TileScale>(plt_float1, 4.0f, 1.0f);
66
67  //Floating platform 2
68  auto plt_float2 = m_ECSScene->CreateSpriteEntity(Vector3D(8.0f, -3.0f, 0.0f),
69
70              Vector3D(4.0f, 1.0f, 1.0f),
71
72              tileTex);
73
74  m_PlatformEntities.push_back(plt_float2);
75  m_ECSScene->GetRegistry().emplace<Components::TileScale>(plt_float2, 4.0f, 1.0f);
76
77  //Central platform
78  auto plt_floatcenter = m_ECSScene->CreateSpriteEntity(Vector3D(0.0f, 2.0f, 0.0f),
79
80              Vector3D(8.0f, 1.0f, 1.0f),
81
82              tileTex);
83
84  m_PlatformEntities.push_back(plt_floatcenter);
85  m_ECSScene->GetRegistry().emplace<Components::TileScale>(plt_floatcenter, 8.0f,
86  1.0f);
87
88  //Spawn enemies
89  SpawnEnemies();
90
91  //Initialize player controller
```

```
85         m_PlayerController = std::make_unique<PlayerController>(m_ECSScene-
>GetRegistry(),
86                                     m_PlayerEntity, *m_World);
87     m_PlayerController->SetSpeed(4.0f);
88
89     //Initialize FPS counter
90     m_FpsCounter = std::make_unique<FpsCounter>();
91 }
```

A.6 — MenuScene::Update

```

1 void MenuScene::Update(float deltaTime){

2     //Keyboard navigation - Down

3         if(Input::IsKeyPressed(SDL_SCANCODE_DOWN) ||
Input::IsKeyPressed(SDL_SCANCODE_S) ||

4             GamepadManager::GetInstance().IsButtonPressed(0,
SDL_GAMEPAD_BUTTON_DPAD_DOWN)){

5         m_SelectedOption = (m_SelectedOption + 1) % static_cast<int>(m_Options.size());

6         Log::Info("Selected: " + m_Options[m_SelectedOption].Label);

7     }

8

9     //Keyboard navigation - Up

10        if(Input::IsKeyPressed(SDL_SCANCODE_UP) ||
Input::IsKeyPressed(SDL_SCANCODE_W) ||

11            GamepadManager::GetInstance().IsButtonPressed(0,
SDL_GAMEPAD_BUTTON_DPAD_UP)){

12        m_SelectedOption = (m_SelectedOption - 1 + static_cast<int>(m_Options.size())) %
static_cast<int>(m_Options.size());

13        Log::Info("Selected: " + m_Options[m_SelectedOption].Label);

14    }

15

16    //Confirm selection with keyboard

```

```

17             if(Input::IsKeyPressed(SDL_SCANCODE_RETURN) ||
Input::IsKeyPressed(SDL_SCANCODE_SPACE) ||

18             GamepadManager::GetInstance().IsButtonPressed(0,
SDL_GAMEPAD_BUTTON_EAST)){

19         HandleSelection();

20     }

21

22     //Mouse click detection

23     if(Input::IsMouseButtonPressed(1)){

24         Vector2D mousePos = Input::GetMousePosition();

25         int winWidth = Input::GetWindowWidth();

26         int winHeight = Input::GetWindowHeight();

27

28         //Convert to world coordinates (projection: -10 to 10)

29         float worldX = (mousePos.x / winWidth) * 20.0f - 10.0f;

30         float worldY = -(mousePos.y / winHeight) * 20.0f + 10.0f;

31

32         for(size_t i = 0; i < m_Options.size(); ++i){

33             const auto& opt = m_Options[i];

34             float halfWidth = (opt.Width / opt.Height) * 1.5f * 0.5f;

35             float halfHeight = 0.75f;

```

```

36
37     float left = opt.Position.x - halfWidth;
38     float right = opt.Position.x + halfWidth;
39     float bottom = opt.Position.y - halfHeight;
40     float top = opt.Position.y + halfHeight;
41
42     if(worldX >= left && worldX <= right && worldY >= bottom && worldY <= top)
43     {
44         m_SelectedOption = static_cast<int>(i);
45         HandleSelection();
46         break;
47     }
48 }
49 }
```

Apêndice B — Diagrama de Classes

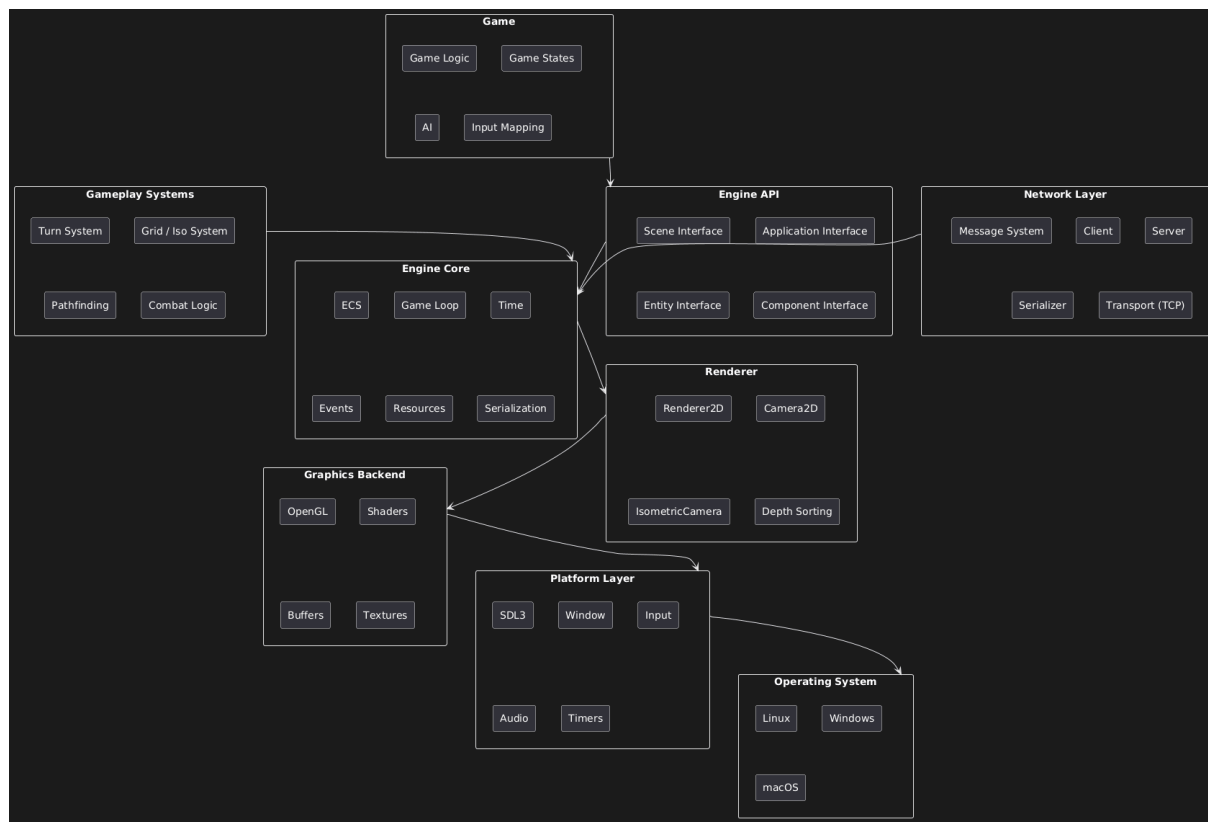


Figura 20- Diagrama UML de classes

Anexos

Anexo A — Documentação Doxygen Gerada

A documentação HTML foi gerada com Doxygen 1.12 e está disponível em `docs/html/index.html`. Inclui todas as classes, métodos e ficheiros do projeto.

Anexo B — Instruções de Compilação

Windows

```
1 cmake -S . -B build\windows -G Ninja
2 ninja -C build\windows
```

Linux

```
1 cmake -S . -B build/linux -G Ninja
2 ninja -C build/linux
```

Anexo C — Referências Consultadas Online

- <https://learnopengl.com>
- <https://wiki.libsdl.org/SDL3>
- <https://gameprogrammingpatterns.com>
- <https://www.khronos.org/opengl/>
- <https://www.youtube.com/watch?v=WficzyoTSsg&list=PLYokS5qr7lSsvgemrTwMSrQsdk4BRqJU6>